

Mémoire de projet de fin d'étude

Filière : Licence Sciences Mathématiques et Informatique

Option : Développement Informatique

Sujet :

Conception et développement d'un système web de
gestion des soutenances et des
jurys

Réalisé par :

ZAITOUNI Nourelislam

TAOUDI EBOURKI Abdelaziz

Soutenu le 19/06/2026, devant le jury :

Pr. AIT DAOUD Mohammed, Encadrant

Dr. SAMMAH Salma, Co-encadrante

Pr. OUCHRA Hafsa, Examinatrice

Dr. AJBAL Khaoula, Examinatrice

À nos parents, pour leur soutien indéfectible.

À nos professeurs, pour leur encadrement et leur savoir.

À Pr. Mohammed AIT DAOUD et Dr. Salma SAMMAH, pour leur encadrement précieux, leur disponibilité et leurs conseils avisés tout au long de ce projet.

À tous ceux qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Remerciements

Nous tenons à exprimer notre profonde gratitude à nos encadrants, **Pr. Mohammed AIT DAOUD** et **Dr. Salma SAMMAH**, pour la confiance qu'ils nous ont témoignée en nous confiant ce sujet, ainsi que pour leur encadrement précieux, leur disponibilité exemplaire et la pertinence de leurs conseils, qui ont été déterminants pour la réalisation et la réussite de ce projet.

Nos remerciements s'adressent également à l'ensemble du corps professoral du département Mathématique Informatique de la Faculté des Sciences Ben M'Sick pour la qualité de la formation dispensée durant ces trois années de licence.

Enfin, nous remercions nos familles et nos proches pour leur patience et leurs encouragements constants tout au long de notre parcours académique.

Résumé

L'organisation des soutenances en milieu universitaire constitue un processus complexe mobilisant de multiples acteurs – étudiants, encadrants, membres de jury et personnel administratif – autour de contraintes temporelles, spatiales et réglementaires. Le présent projet vise à concevoir et développer une application web complète pour la gestion centralisée, automatisée et fiable des soutenances et des jurys au sein d'un établissement universitaire. Le système repose sur une architecture moderne à trois couches : un frontend React, un backend Spring Boot et une base de données MySQL. Il couvre l'ensemble du cycle de vie d'une soutenance, de la planification des créneaux et de l'affectation des jurys jusqu'à la saisie des évaluations et la génération automatique des documents administratifs (convocations, procès-verbaux, fiches d'évaluation). Une attention particulière est portée à la détection des conflits horaires et à l'équilibrage de la charge des enseignants. Ce rapport présente l'étude de l'existant, l'analyse des besoins, la conception UML, l'architecture technique et la modélisation de la base de données du système.

Mots-clés : Gestion de soutenances, planification, jury, évaluation, Spring Boot, React, JWT, UML, système d'information universitaire.

Abstract

Organizing thesis defenses in a university environment is a complex process involving multiple actors – students, supervisors, jury members, and administrative staff – around temporal, spatial and regulatory constraints. This project aims to design and develop a complete web application for the centralized, automated and reliable management of defenses and juries at a university institution. The system is based on a modern three-tier architecture : a React frontend, a Spring Boot backend, and a MySQL database. It covers the entire lifecycle of a defense, from slot planning and jury assignment to evaluation entry and automatic generation of administrative documents (summons, minutes, evaluation sheets). Special attention is given to conflict detection and teacher workload balancing. This report presents the study of the current system, requirements analysis, UML design, technical architecture, and database modeling.

Keywords : Defense management, planning, jury, evaluation, Spring Boot, React, JWT, UML, university information system.

Table des matières

Remerciements	3
Résumé	4
Abstract	4
Liste des figures	7

Liste des tableaux	8
Introduction Générale	10
Contexte général	10
Problématique	10
Objectifs du projet	11
Méthodologie suivie	11
Structure du rapport	11
1 Présentation de l'organisme d'accueil	13
1.1 Introduction	13
1.2 Présentation générale de l'organisme	13
1.2.1 Historique et fiche signalétique	13
1.2.2 Missions et activités	14
1.2.3 Secteurs d'intervention	14
1.3 Structure organisationnelle	15
1.3.1 Organigramme	15
1.3.2 Description des départements	16
1.4 Environnement du stage	16
1.4.1 Département d'accueil	16
1.4.2 Contexte du projet au sein de l'organisme	16
1.4.3 Encadrement et organisation du stage	17
1.5 Conclusion	17
2 Étude de l'existant et technologies utilisées	18
2.1 Introduction	18
2.2 Analyse de l'existant	18
2.2.1 Description du système actuel	18
2.2.2 Processus actuel détaillé	18
2.2.3 Impact quantifié du système actuel	21
2.2.4 Comparaison processus actuel vs. processus souhaité	22
2.2.5 Analyse critique	22
2.3 Étude des solutions similaires	23
2.3.1 Systèmes universitaires généraux (ERP)	23
2.3.2 Outils de planification générique	23
2.3.3 Solutions open source (GitHub)	24
2.3.4 Synthèse et positionnement	24
2.4 Choix technologiques	25
2.4.1 Frontend	25
2.4.2 Backend	25
2.4.3 Base de données	26
2.4.4 Outils de développement	26
2.4.5 Outils de qualité de code	27
2.5 Architecture générale retenue	27
2.5.1 Philosophie API-first	27
2.5.2 Organisation multi-dépôts	28
2.5.3 Stratégie de déploiement	28
2.6 Conclusion	28

3	Analyse des besoins et conception du système	30
3.1	Introduction	30
3.2	Présentation générale du projet	30
3.3	Cahier des charges	30
3.3.1	Acteurs du système	30
3.3.2	Besoins fonctionnels	31
3.3.3	Besoins non fonctionnels	32
3.4	Règles de gestion	33
3.5	Conception UML	34
3.5.1	Diagrammes de cas d'utilisation	34
3.5.2	Diagramme de classes	38
3.5.3	Diagrammes de séquence	39
3.5.4	Diagrammes d'activité et de machine d'états	42
3.6	Architecture logicielle	46
3.6.1	Backend (API REST)	46
3.6.2	Frontend (SPA)	46
3.6.3	Sécurité et contrôle d'accès	46
3.7	Modélisation de la base de données	46
3.8	Normes et standards respectés	48
3.9	Conclusion	48
4	Réalisation et développement	49
4.1	Introduction	49
4.2	Environnement de développement	49
4.2.1	Matériel et Logiciels	49
4.2.2	Workflow de développement	50
4.3	Architecture technique du système	50
4.3.1	Le Backend : API REST Spring Boot	50
4.3.2	Le Frontend : SPA React	51
4.4	Implémentation des modules fonctionnels	51
4.4.1	Module Authentification et Sécurité	51
4.4.2	Le Defense Designer : Planification Interactive	52
4.4.3	Le Moteur de Détection de Conflits	54
4.4.4	Module d'Évaluation et Notation	56
4.4.5	Génération Documentaire et Stratégie d'Impression	56
4.4.6	Système de Notifications	57
4.4.7	Système d'Audit	57
4.4.8	Module de Configuration Administrative	58
4.4.9	Fonctionnalités Étudiant et Enseignant	58
4.5	Interface utilisateur et navigation	58
4.6	Bonnes pratiques adoptées	60
4.6.1	Tests automatisés	60
4.6.2	Hooks de pré-commit	60
4.6.3	Conventions Git et GitHub	60
4.6.4	Pipeline CI/CD	60
4.7	Difficultés rencontrées et solutions	61
4.8	Conclusion	61

5	Stratégie de test et validation	62
5.1	Introduction	62
5.2	Stratégie de test	62
5.2.1	Tests unitaires et d'intégration (Backend)	62
5.2.2	Tests de l'interface utilisateur (Frontend)	63
5.2.3	Tests de bout en bout (E2E)	64
5.3	Scénarios de validation et résultats	64
5.4	Métriques de qualité	65
5.5	Évaluation globale et limites	66
5.6	Conclusion	66
	Conclusion Générale	68
	Bibliographie	69

Table des figures

1	Organigramme simplifié de la FSBM	15
2	Processus actuel – Phase 1 : Planification	19
3	Processus actuel – Phase 2 : Exécution	20
4	Architecture en composants du système	28
5	Hiérarchie générale des acteurs	35
6	Cas d'utilisation : Administration du système	36
7	Cas d'utilisation : Coordination et Defense Designer	37
8	Cas d'utilisation : Enseignant / Membre de jury	38
9	Cas d'utilisation : Étudiant	38
10	Diagramme de classes global du système	39
11	Séquence de planification interactive via le Defense Designer	40
12	Séquence d'authentification via JWT	41
13	Séquence de soumission d'une évaluation	41
14	Logique de validation des 8 règles de gestion (RG1-RG8)	42
15	Processus d'évaluation et de notation d'une soutenance	43
16	Machine à états du cycle de vie d'une soutenance	45
17	Modèle Logique de Données (MLD) synchronisé	47

Liste des tableaux

1	Fiche signalétique de la FSBM (effectif publié en 2015–2016)	14
2	Estimation du temps consacré par étape lors d'une session de 50 soutenances	21
3	Comparaison entre le processus actuel et le processus proposé	22
4	Analyse des systèmes ERP académiques	23
5	Analyse des outils de planification génériques	23
6	Analyse de projets open source similaires	24
7	Synthèse comparative des solutions	24
8	Répartition des fonctionnalités par acteur	31
9	Correspondance entre les règles de gestion et leurs implémentations	34
10	Outils et versions utilisés dans le projet	49

11	Cartographie complète des pages de l'application	59
12	Matrice de validation des fonctionnalités critiques	65
13	Métriques de qualité du système	66

Introduction Générale

Contexte général

L'organisation des soutenances en milieu universitaire constitue une activité critique dont la qualité conditionne directement le bon déroulement du parcours académique des étudiants. Chaque année, les départements universitaires doivent coordonner un ensemble complexe de tâches : la planification des créneaux de soutenance, la constitution des jurys, la réservation des salles, la production des documents administratifs, la saisie des évaluations et la consolidation des résultats. Cette coordination implique la mobilisation simultanée de multiples acteurs – étudiants, encadrants, membres de jury, coordinateurs de département et services administratifs – chacun disposant de contraintes spécifiques et de responsabilités distinctes.

La Faculté des Sciences Ben M'Sick (FSBM) de l'Université Hassan II de Casablanca, à travers son département de Mathématiques, Informatique et Physique (MIP), fait face à cette réalité avec des effectifs importants : plusieurs milliers d'étudiants répartis en plusieurs filières de licence et de master, encadrés par un corps enseignant diversifié. L'organisation des soutenances de fin de cycle – licences et masters – mobilise l'ensemble du département pendant plusieurs semaines, avec des centaines de projets à planifier simultanément. Dans ce contexte, les processus traditionnels reposant sur des tableurs Excel, des échanges d'e-mails massifs et des documents papier engendrent une perte d'efficacité significative, des erreurs de planification fréquentes et une traçabilité insuffisante des décisions. La nécessité de moderniser ces outils s'impose avec une urgence croissante, motivée à la fois par les exigences de qualité pédagogique et par la volonté de offrir aux étudiants et enseignants une expérience numérique fiable et ergonomique.

Problématique

La coordination des soutenances dans un département universitaire fait face à une complexité croissante liée à la multiplicité des contraintes à respecter simultanément. Les disponibilités des enseignants, qui interviennent à la fois comme encadrants de projets et comme membres de jury, doivent être prises en compte pour éviter les doubles affectations horaires et garantir que chaque jury dispose des compétences nécessaires. La réservation des salles, dont les capacités varient et dont la disponibilité est limitée, nécessite une allocation cohérente avec le nombre de participants attendus et les équipements requis. La gestion des sessions de soutenance, avec leurs phases de constitution, de planification et d'exécution, ajoute une dimension temporelle à ces contraintes spatiales et humaines.

Par ailleurs, la production des documents administratifs accompagnant chaque soutenance – convocations officielles, fiches d'évaluation standardisées, procès-verbaux de délibération – représente une charge de travail récurrente et chronophage, souvent réalisée manuellement avec des modèles hétérogènes. La saisie et le suivi des évaluations, la transmission des notes aux services compétents et la génération des rapports de synthèse complètent ce tableau d'inefficacités. Les erreurs humaines – notes mal attribuées, jurys incomplets, créneaux en conflit – ont des conséquences directes sur les étudiants et nécessitent des interventions correctives coûteuses.

Le cœur de la problématique réside donc dans l'absence d'un outil unifié capable de gérer l'ensemble de ces aspects de manière intégrée et automatisée. L'organisation manuelle actuelle, basée sur des outils génériques non adaptés à cette finalité spécifique, génère des inefficacités structurelles, des erreurs évitables et une absence de traçabilité des décisions organisationnelles. C'est dans ce contexte que s'inscrit le présent projet, qui vise à concevoir et développer

une plateforme web de gestion des soutenances, répondant aux exigences des établissements universitaires.

Objectifs du projet

Objectif principal: Développer une application web complète et fonctionnelle permettant la gestion centralisée, automatisée et fiable de l'organisation des soutenances et des jurys au sein d'un établissement universitaire.

Cette vision générale se décline en plusieurs objectifs spécifiques, organisés autour de quatre axes fonctionnels majeurs. Le premier axe concerne la gestion rigoureuse des acteurs académiques : étudiants, encadrants, membres de jury et responsables pédagogiques, avec un contrôle granulaire des accès basé sur les rôles. Le deuxième axe porte sur la planification des soutenances, intégrant la gestion des calendriers, des salles et des sessions, ainsi que l'affectation automatique des jurys tout en détectant les conflits de manière proactive via huit règles de gestion. Le troisième axe vise l'automatisation de la production documentaire : convocations, fiches d'évaluation et procès-verbaux, avec support de l'impression A4 et du mode portrait. Le quatrième axe concerne le module d'évaluation, permettant une saisie fiable des notes avec calcul automatique des moyennes pondérées et enregistrement formel des délibérations.

Méthodologie suivie

La réalisation de ce projet a été menée selon une démarche structurée en quatre phases successives, conformément aux méthodologies classiques de génie logiciel adaptées au contexte universitaire. Le processus a débuté par une phase d'analyse de l'existant, visant à étudier les mécanismes actuels d'organisation des soutenances au sein du département MIP, à en identifier les limites opérationnelles et à formaliser les besoins réels via des entretiens avec les coordinateurs et enseignants impliqués. Cette étape a posé les bases pour la phase de conception, consacrée à la modélisation UML – incluant les cas d'utilisation, les diagrammes de classes, de séquence et d'activité – ainsi qu'à la définition de l'architecture technique et de la modélisation de la base de données relationnelle.

S'en est suivie la phase de réalisation, focalisée sur l'implémentation effective du backend Spring Boot et du frontend React, selon une approche itérative permettant de valider les fonctionnalités au fil du développement. La phase finale de tests et de validation a permis de garantir la conformité fonctionnelle et la robustesse du système, en combinant des tests unitaires (86 classes Java, 91 fichiers TypeScript), des tests d'intégration et des tests de bout en bout (environ 65 scénarios Playwright) contre l'API réelle. Cette méthodologie multi-niveaux a permis de détecter et corriger les anomalies avant le déploiement, assurant ainsi la qualité du système livré.

Structure du rapport

Le présent rapport est organisé en cinq chapitres distincts, chaque chapitre correspondant à une phase du projet ou à un axe d'analyse spécifique. Le premier chapitre présente le contexte du stage au sein de la faculté d'accueil, en décrivant l'organisme d'accueil, son organisation et son infrastructure. Le deuxième chapitre dresse un état des lieux du système actuel, examine les solutions comparables sur le marché et justifie les choix technologiques effectués en fonction des besoins identifiés. Le troisième chapitre détaille l'analyse des besoins fonctionnels et non

fonctionnels, et présente la conception complète du système via les diagrammes UML et le modèle de données. Le quatrième chapitre décrit la réalisation technique, en présentant les choix d'architecture, les défis rencontrés et les solutions apportées. Enfin, le cinquième chapitre expose la stratégie de test, les métriques de qualité et les modalités de validation du système, avant qu'une conclusion générale ne dresse le bilan du travail réalisé et ne trace les perspectives d'évolution du projet.

1 Présentation de l'organisme d'accueil

1.1 Introduction

Ce chapitre présente l'environnement dans lequel s'est déroulé notre projet de fin d'études. Il décrit la Faculté des Sciences Ben M'Sick (FSBM) – son histoire, ses missions, sa structure organisationnelle – ainsi que le département d'accueil et le cadre spécifique du stage. Cette contextualisation permet de mieux comprendre les enjeux institutionnels et pédagogiques qui ont motivé la réalisation de notre système de gestion des soutenances. Comprendre l'organisme d'accueil est essentiel pour appréhender les contraintes réelles auxquelles fait face le département MIP dans l'organisation de ses soutenances, et pour justifier les choix techniques adoptés dans notre solution.

1.2 Présentation générale de l'organisme

1.2.1 Historique et fiche signalétique

La Faculté des Sciences Ben M'Sick (FSBM) est l'une des composantes de l'Université Hassan II de Casablanca (UH2C). Elle a ouvert ses portes en **1984**, dans le cadre d'une politique de décentralisation des institutions universitaires au Maroc, et est rattachée à l'Université Hassan II – Mohammedia – Casablanca, qui regroupe six établissements universitaires répartis sur deux campus : le campus de Casablanca et le campus de Mohammedia.

Dès son ouverture, la FSBM a accordé un intérêt particulier au développement de la recherche scientifique parallèlement à sa mission d'enseignement et de formation. De 1984 à 2003, elle a dispensé divers formations de type premier cycle (DEUG) et deuxième cycle (Licences sciences : Bac+4) dans divers spécialités. À partir de 1989, les premières formations de 3ème cycle (CEA et DES) ont commencé grâce à la mise en place d'un certain nombre d'équipes et de laboratoires de recherche. En 1997, avec la création des UFR (Unités de Formation et de Recherche), la nouvelle réorganisation de la formation à et par la recherche a donné lieu à de nouveaux regroupements de chercheurs autour de nouvelles thématiques. Depuis 2003, la faculté dispense une formation modulaire et semestrielle dans le cadre de la réforme pédagogique de l'enseignement supérieur conformément au système LMD (Licence-Master-Doctorat).

En 2008, suite à la réorganisation du cycle doctorat, la faculté a mis en place le Centre d'Études Doctorales (CED) : « Sciences et applications », adossé à l'ensemble des structures de recherches accréditées par l'université. Le 1er septembre 2014, une étape majeure a été franchie avec la fusion de l'Université Hassan II Mohammedia et de l'Université Hassan II Casablanca, donnant naissance à l'Université Hassan II de Casablanca (UH2C). Cette fusion avait pour objectifs d'optimiser les ressources, d'harmoniser les programmes académiques et de renforcer la compétitivité de l'université au niveau national et international.

Information	Détail
Nom officiel	Faculté des Sciences Ben M'Sick (FSBM)
Université de rattachement	Université Hassan II de Casablanca (UH2C)
Adresse	Avenue Commandant Driss El Harti, Ben M'Sick, Casablanca
Année de création	1984
Type	Établissement public d'enseignement supérieur
Domaines	Sciences exactes, Sciences et technologies
Nombre de départements	6 (Mathématiques, Informatique, Physique, Chimie, Biologie, Géologie)
Licences fondamentales	6 parcours (selon l'offre de formation affichée par la FSBM)
Masters	18 spécialités (selon l'offre de formation affichée par la FSBM)
Structures de recherche	23 (19 laboratoires, 2 centres, 1 observatoire, 1 plateforme)
Effectif étudiant (2015–2016)	≈ 11 500 étudiants

TABLE 1 – Fiche signalétique de la FSBM (effectif publié en 2015–2016)

1.2.2 Missions et activités

La FSBM remplit plusieurs missions fondamentales qui structurent son fonctionnement au quotidien. Sur le plan de la formation, elle assure la formation initiale en dispensant un enseignement supérieur de qualité dans les disciplines scientifiques fondamentales et appliquées aux niveaux Licence, Master et Doctorat. L'offre de formation couvre un large éventail de spécialités, des mathématiques pures à l'informatique appliquée, en passant par les sciences physiques et de la vie, permettant aux étudiants de construire un parcours académique solide et diversifié.

Parallèlement à sa mission d'enseignement, la faculté développe une activité intense de recherche scientifique au sein de ses 23 structures de recherche, qui incluent 19 laboratoires, 2 centres de recherche, un observatoire et une plateforme technologique (PINTECH). Ces unités de recherche, souvent labellisées par le Centre National pour la Recherche Scientifique et Technique (CNRST), travaillent sur des projets nationaux et internationaux, contribuant ainsi à l'avancement des connaissances dans des domaines tels que les mathématiques appliquées, l'intelligence artificielle, la modélisation physique, la biotechnologie et la chimie des matériaux. La présence de ces laboratoires enrichit l'environnement pédagogique en permettant aux étudiants de s'initier à la recherche dès le cycle de licence.

L'établissement se consacre également à préparer activement ses étudiants à leur insertion dans le monde professionnel à travers des formations adaptées, des stages pratiques et des partenariats avec des entreprises du secteur technologique. La faculté a noué des relations de coopération internationale à travers la signature de conventions et accords avec des universités, des institutions universitaires, des centres et des laboratoires de recherche au Maroc et à l'étranger. Enfin, elle contribue au rayonnement académique, scientifique et technologique de la région et du pays en organisant des colloques, des journées scientifiques et des échanges académiques avec des institutions partenaires.

1.2.3 Secteurs d'intervention

La faculté couvre un spectre étendu de disciplines scientifiques organisées en six départements principaux. Le département de Mathématiques propose les licences en Mathématiques

Fondamentales et Appliquées, en plus des masters spécialisés en analyse, algèbre, probabilités-statistiques et mathématiques appliquées. Le département d'Informatique est responsable de la filière Licence MIP (Mathématiques, Informatique et Physique) et de plusieurs masters en informatique, couvrant des domaines tels que le développement logiciel, les bases de données, les réseaux, l'intelligence artificielle et le traitement de l'information.

Les sciences physiques sont représentées par le département de Physique, qui forme les étudiants aux fondements de la mécanique, de l'électromagnétisme, de la thermodynamique et de l'optique. Le département de Chimie offre des formations en chimie générale, organique, analytique et des matériaux. Les sciences de la vie sont traitées par le département de Biologie, avec des spécialités en biologie moléculaire, biotechnologie, écologie et sciences de la santé. Enfin, le département de Géologie couvre les sciences de la terre, la géophysique, la géomatique et la gestion de l'environnement. Cette diversité disciplinaire fait de la FSBM un établissement de premier plan dans le paysage de l'enseignement supérieur scientifique marocain, formant chaque année des milliers de diplômés susceptibles d'occuper des postes clés dans les secteurs académique, industriel et technologique.

1.3 Structure organisationnelle

1.3.1 Organigramme

La FSBM est administrée par un Doyen, assisté par des Vice-Doyens et un Secrétaire Général. Sa structure s'articule autour de plusieurs organes décisionnels et entités fonctionnelles, au premier rang desquels figure le Conseil de faculté, instance délibérante composée d'enseignants-chercheurs, d'étudiants et de représentants administratifs. Le fonctionnement académique est encadré par des commissions pédagogiques chargées de l'organisation et du suivi des filières.

La structure de gouvernance de la faculté peut être décomposée en trois niveaux. Au niveau stratégique, le Conseil de faculté définit les orientations générales, valide les programmes de formation et approuve le budget annuel. Au niveau opérationnel, le Doyen et les Vice-Doyens coordonnent les activités pédagogiques, scientifiques et administratives. Au niveau départemental, les chefs de département et leurs conseils assurent la gestion quotidienne des filières, la coordination des emplois du temps et le suivi pédagogique des étudiants. La faculté s'appuie également sur des laboratoires de recherche ainsi que des services administratifs essentiels, notamment la scolarité, les ressources humaines, les finances et la bibliothèque.

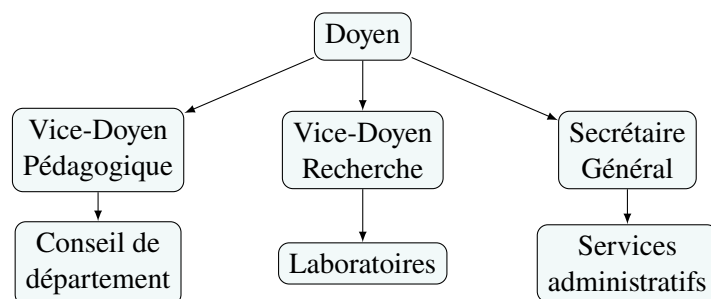


FIGURE 1 – Organigramme simplifié de la FSBM

1.3.2 Description des départements

La structuration pédagogique repose sur six départements, chacun responsable d'une ou plusieurs filières. Le Département de Mathématiques propose les licences en Mathématiques Fondamentales et Appliquées, en plus des masters spécialisés. Le Département d'Informatique est responsable de la filière Licence MIP (Mathématiques, Informatique et Physique) et de plusieurs masters en informatique, incluant des spécialités telles que Data Sciences et Big Data, Intelligence Artificielle, et Traitement de l'Information. Les autres domaines sont couverts par le Département de Physique, le Département de Chimie, le Département de Biologie et le Département de Géologie, qui offrent chacun des formations spécialisées.

Ces départements sont dirigés par des chefs élus, assistés par un conseil de département, assurant la coordination des enseignements et des emplois du temps. Chaque département dispose de son propre ensemble de salles de cours, de salles informatiques et de laboratoires, lui permettant d'organiser ses activités de formation de manière autonome. Le département d'Informatique, en particulier, dispose de plusieurs salles équipées de postes de travail connectés, indispensables aux enseignements pratiques en développement logiciel et en administration de bases de données.

1.4 Environnement du stage

1.4.1 Département d'accueil

Notre projet de fin d'études s'est déroulé au sein de la filière **Licence Fondamentale MIP (Mathématiques, Informatique et Physique)** du Département d'Informatique. Cette filière, créée pour former des étudiants polyvalents possédant une solide base en mathématiques et une maîtrise des technologies de l'information, a été conçue en réponse à la demande croissante du secteur technologique marocain pour des profils à l'interface entre les mathématiques et l'informatique. La formation couvre des domaines variés tels que le développement logiciel, les bases de données, les réseaux, l'algorithmique avancée et l'intelligence artificielle, tout en maintenant un socle mathématique rigoureux incluant l'analyse, l'algèbre, les probabilités et les statistiques.

Le corps enseignant de la filière est composé de professeurs de l'enseignement supérieur, de professeurs habilités et de professeurs assistants, couvrant l'ensemble des spécialités du programme. Cette diversité de profils garantit aux étudiants un encadrement de qualité dans chaque domaine d'études, depuis les fondements théoriques jusqu'aux applications pratiques les plus récentes.

1.4.2 Contexte du projet au sein de l'organisme

Le projet de système de gestion des soutenances et des jurys s'inscrit dans une volonté de modernisation des processus administratifs et pédagogiques au sein de la faculté. Actuellement, l'organisation des soutenances – qu'il s'agisse des soutenances de PFE, de mémoires de master ou de thèses – repose sur des méthodes manuelles et des outils bureautiques génériques. Les coordinateurs utilisent des fichiers Excel pour suivre les projets, des emails pour communiquer avec les enseignants et les étudiants, et des supports papier pour les convocations et les procès-verbaux. Cette organisation, bien que fonctionnelle pour un petit nombre de soutenances, montre rapidement ses limites lorsque le volume d'étudiants augmente.

Le département MIP, confronté à un volume croissant d'étudiants – avec plusieurs centaines d'étudiants par promotion – et à une complexité accrue dans la coordination des jurys, a identifié

le besoin d'un outil informatique dédié. Les problèmes les plus fréquemment rencontrés incluent les conflits d'horaires entre enseignants, les erreurs de planification liées à la manipulation manuelle de grandes quantités de données, l'absence de traçabilité des décisions prises lors de l'organisation des sessions, et la lourdeur de la génération des documents administratifs (convocations, fiches d'évaluation, procès-verbaux). Ces difficultés sont amplifiées lors des sessions de rattrapage, où la planification doit être reprise en tenant compte des contraintes de disponibilité des enseignants et des salles déjà partiellement occupées.

Ce projet a donc été proposé par **Pr. Mohammed AIT DAOUD** dans le cadre des projets de fin d'études de la licence MIP, avec pour objectif de fournir une solution concrète et opérationnelle répondant aux besoins réels du département. Le choix de ce sujet reflète la volonté de la faculté de s'appuyer sur les compétences de ses propres étudiants pour moderniser ses outils de gestion, tout en offrant aux futurs diplômés l'opportunité de travailler sur un projet à dimension réelle.

1.4.3 Encadrement et organisation du stage

Le stage s'est déroulé sous la double supervision de **Pr. Mohammed AIT DAOUD**, professeur au département d'Informatique de la FSBM, et de **Dr. Salma SAMMAH**, également membre du corps enseignant de la filière MIP. Cette structure d'encadrement a permis de bénéficier de orientations complémentaires : une vision architecturale et méthodologique du côté de Pr. AIT DAOUD, et un regard plus orienté vers les aspects fonctionnels et la validation métier du côté de Dr. SAMMAH.

Les deux étudiants responsables du projet se sont réparti les tâches en fonction de leurs centres d'intérêt et de leurs compétences complémentaires. L'un s'est principalement concentré sur le développement du backend (API REST, logique métier, sécurité) et l'autre sur le frontend (interface utilisateur, composants React, intégration des designs). Cette répartition a permis un développement parallèle efficace, chaque étudiant travaillant de manière autonome tout en maintenant une coordination étroite via des points réguliers et un dépôt Git commun.

Le suivi du projet s'est organisé autour de réunions hebdomadaires avec les encadrants, au cours desquelles l'avancement était présenté, les difficultés étaient discutées et les orientations techniques étaient validées. Cette organisation agile a permis d'ajuster le périmètre du projet en cours de développement, en priorisant les fonctionnalités critiques (planification, détection de conflits) par rapport aux fonctionnalités secondaires qui pouvaient être reportées.

1.5 Conclusion

La Faculté des Sciences Ben M'Sick constitue un cadre académique riche et diversifié, comptant plusieurs milliers d'étudiants et offrant des formations scientifiques de qualité dans un large éventail de disciplines. Le département MIP, par sa proximité avec les technologies de l'information et par le volume croissant de ses étudiants, est confronté à des défis croissants dans l'organisation de ses soutenances. Ce constat, combiné à la volonté de modernisation de la faculté – qui a déjà engagé sa transformation numérique à travers le système LMD et la plateforme PINTECH – et à l'expertise technique disponible au sein de son corps étudiant, a créé le contexte idéal pour le développement d'une solution numérique de gestion des soutenances. Ce contexte a directement inspiré les choix technologiques et architecturaux qui seront présentés dans les chapitres suivants.

2 Étude de l'existant et technologies utilisées

2.1 Introduction

Ce chapitre présente une analyse approfondie du système actuel d'organisation des soutenances au sein de la faculté, identifie ses lacunes et compare les solutions existantes sur le marché. Il détaille ensuite les choix technologiques retenus pour le développement de notre application en justifiant chaque décision par une analyse des alternatives envisagées. Enfin, l'architecture générale du système est présentée, incluant la stratégie de déploiement et l'organisation du code source.

2.2 Analyse de l'existant

2.2.1 Description du système actuel

L'organisation actuelle des soutenances repose sur une combinaison d'outils bureautiques génériques et de processus manuels qui, bien que fonctionnels pour un volume limité de soutenances, montrent rapidement leurs limites face à la croissance des effectifs. Le coordinateur utilise principalement des tableurs Excel pour le suivi des étudiants, des projets et des plannings préliminaires. La communication avec les enseignants et les étudiants se fait par courrier électronique, notamment pour la collecte des disponibilités et l'envoi des convocations. Les salles sont gérées via des services tiers, souvent sans visibilité directe sur les occupations en temps réel. Enfin, l'évaluation des étudiants repose sur une saisie manuelle des notes sur fiches imprimées, suivie d'une re-saisie numérique pour le calcul des moyennes.

Cette organisation, bien qu'ayant le mérite de ne nécessiter aucune infrastructure informatique dédiée et de permettre une certaine flexibilité d'adaptation, engendre des problèmes récurrents qui impactent directement la qualité de l'organisation des soutenances.

2.2.2 Processus actuel détaillé

L'organisation d'une session de soutenances suit un cheminement complexe divisé en deux grandes phases opérationnelles.

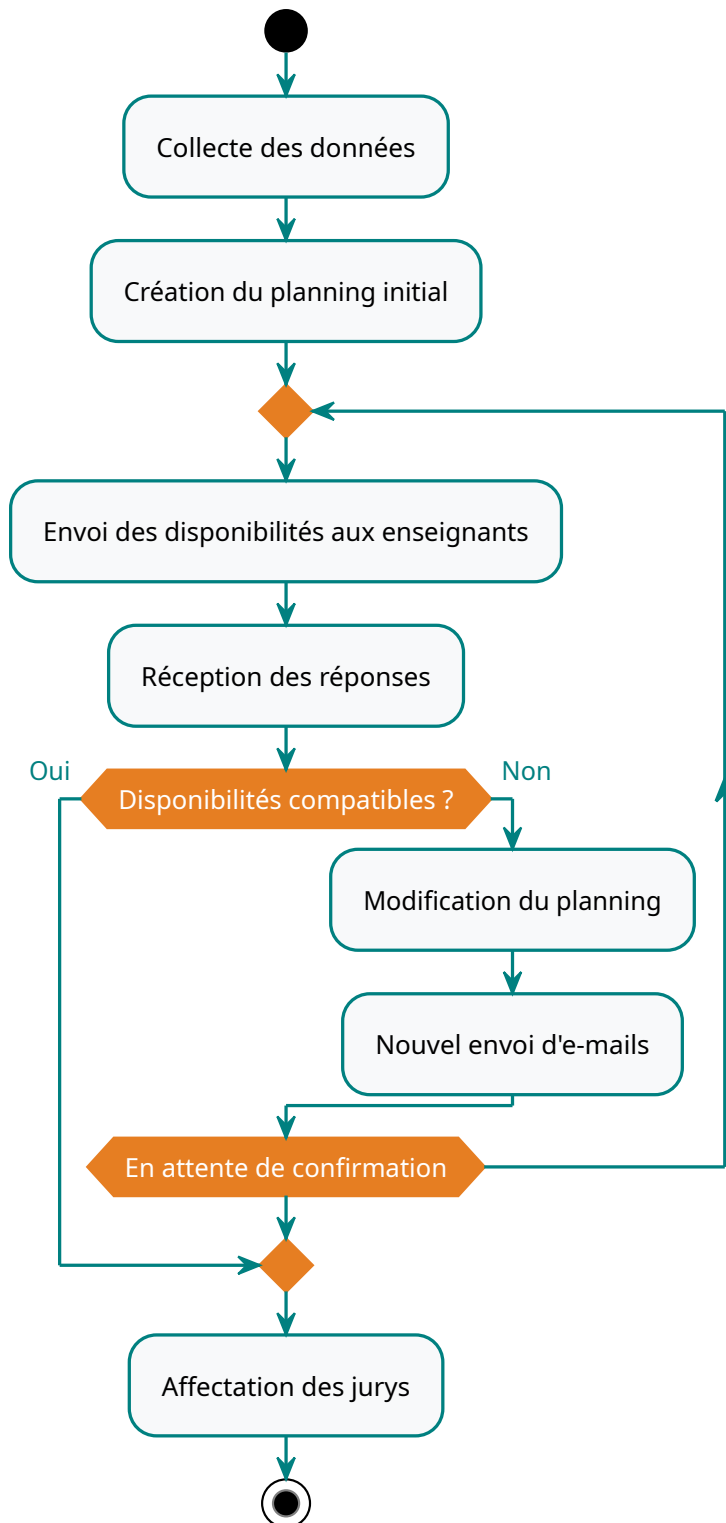


FIGURE 2 – Processus actuel – Phase 1 : Planification

La 2 illustre la nature intrinsèquement itérative de la phase de planification. Elle repose sur une communication asynchrone par e-mail qui génère fréquemment des goulots d'étranglement : la centralisation manuelle des informations (étudiants, sujets, encadrants) et la négociation des créneaux imposent de multiples allers-retours avant d'aboutir à un planning acceptable. Le coordinateur doit collecter les disponibilités de chaque enseignant par e-mail, puis vérifier manuellement que les créneaux proposés ne créent pas de conflits. Cette vérification, réalisée

sur des tableurs Excel, est particulièrement fastidieuse lorsque le nombre d'enseignants et de projets dépasse une dizaine.

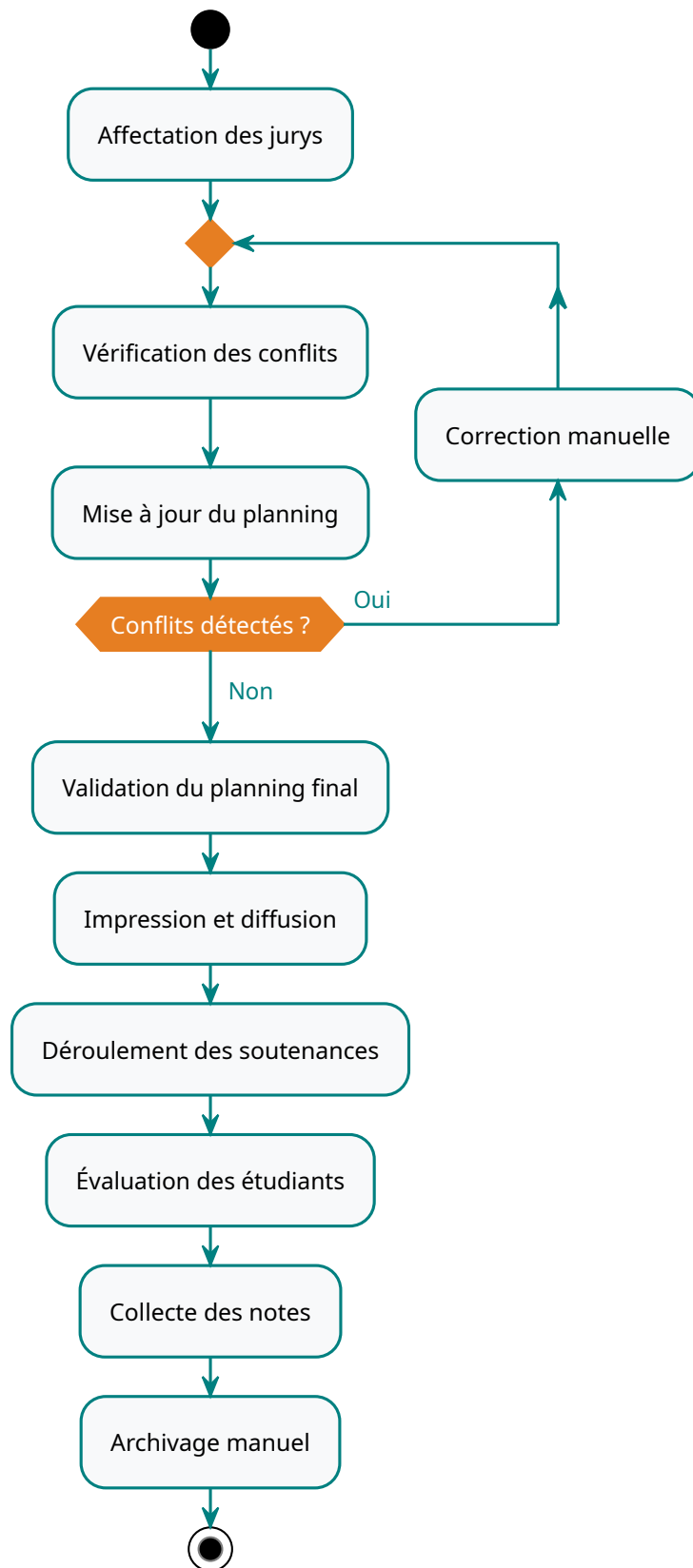


FIGURE 3 – Processus actuel – Phase 2 : Exécution

La 3 détaille la transition vers la phase d'exécution. Cette étape est critique car elle comporte une vérification manuelle des conflits (double affectation d'un enseignant ou d'une salle) avant la diffusion officielle du planning. Le cycle se termine par une collecte physique des notes sur des fiches imprimées, suivie d'une ressaisie numérique pour le calcul des moyennes, et un archivage souffrant d'un manque de centralisation numérique. Les procès-verbaux sont rédigés manuellement et signés physiquement, rendant la traçabilité des décisions particulièrement difficile.

2.2.3 Impact quantifié du système actuel

Pour mesurer concrètement les limites du système actuel, nous avons estimé le temps consacré à chaque étape du processus lors d'une session type d'environ 50 soutenances. Ces estimations sont basées sur les retours d'expérience du coordinateur du département MIP.

Étape du processus	Durée estimée	Problème principal
Collecte des disponibilités enseignants	2–3 semaines	Communication asynchrone par e-mail, relances multiples
Centralisation des données projets	2–3 jours	Saisie manuelle dans Excel, risques d'erreurs de transcription
Vérification des conflits	4–6 heures	Vérification manuelle croisée sur tableur, conflits non détectés
Affectation des salles	1–2 heures	Absence de visibilité en temps réel sur les occupations
Rédaction des convocations	1 jour	Mise en page manuelle pour chaque enseignant et étudiant
Saisie des notes	1–2 jours par session	Resaisie manuelle à partir des fiches papier
Calcul des moyennes	1–2 heures	Formules Excel sensibles aux erreurs de saisie
Rédaction des PV	1–2 jours	Rédaction manuelle, signature physique
Archivage	1 jour	Fichiers dispersés, pas de centralisation
Total estimé	≈ 4–6 semaines	Charge principalement sur le coordinateur

TABLE 2 – Estimation du temps consacré par étape lors d'une session de 50 soutenances

Le tableau 2 met en évidence un constat alarmant : la phase de planification à elle seule représente environ 70% du temps total consacré à l'organisation d'une session, avec une charge particulièrement lourde sur le coordinateur. Les erreurs de planification, souvent détectées uniquement le jour de la soutenance, provoquent des annulations de dernière minute qui nuisent à la réputation de la faculté et au moral des étudiants.

2.2.4 Comparaison processus actuel vs. processus souhaité

Le tableau 3 confronte chaque étape du processus actuel à la solution que notre système propose pour y remédier.

Étape	Méthode actuelle	Problème	Solution proposée
Collecte disponibilités	E-mails individuels	Pas de centralisation	Calendrier interactif dans l'app
Suivi des projets	Fichiers Excel	Pas de versioning	Base de données centralisée
Vérification conflits	Vérification manuelle	Conflits non détectés	Moteur de détection automatique
Affectation jurys	Négociation par e-mail	Lenteur, erreurs	Constitution assistée par templates
Planification créneaux	Tableur Excel	Pas de visualisation	Defense Designer (glisser-déposer)
Génération convocations	Mise en page manuelle	Lenteur, incohérences	Génération automatique A4
Saisie des notes	Fiches papier	Resaisie, erreurs	Saisie numérique par le jury
Calcul des moyennes	Formules Excel	Erreurs de saisie	Calcul automatique pondéré
Archivage	Fichiers dispersés	Perte de données	Stockage centralisé et traçable

TABLE 3 – Comparaison entre le processus actuel et le processus proposé

2.2.5 Analyse critique

Le système actuel présente certains avantages dans des contextes de très petite taille : flexibilité d'adaptation à des cas particuliers, absence de dépendance technologique et charge de travail acceptable pour un volume très limité de soutenances. Cependant, les inconvénients deviennent prépondérants dès que le volume augmente. Les erreurs de planification sont fréquentes et souvent détectées tardivement, provoquant des annulations de dernière minute qui compromettent la crédibilité de l'organisation. La traçabilité des décisions est faible, rendant difficile l'identification des responsabilités en cas de problème. La charge du coordinateur est considérable en fin de semestre, au moment même où les exigences pédagogiques sont les plus élevées. Enfin, la ressaisie répétée des données multiplie les risques d'erreurs de transcription, notamment lors du calcul des moyennes pondérées.

Ces constats justifient pleinement le développement d'un système d'information dédié capable d'automatiser les tâches répétitives, de garantir l'intégrité des données et de réduire la charge de travail du coordinateur tout en améliorant la qualité du service rendu aux étudiants et aux enseignants.

2.3 Étude des solutions similaires

Plusieurs solutions logicielles existent sur le marché ou sous forme de projets open source. Nous avons analysé trois catégories de solutions pour comprendre les forces et faiblesses de chaque approche et positionner notre projet de manière pertinente.

2.3.1 Systèmes universitaires généraux (ERP)

Solution	Description	Adéquation
Apogée (Amue)	Système de gestion des étudiants utilisé dans les universités françaises	Trop généraliste ; gestion des soutenances non native
Aurion	ERP académique couvrant scolarité, stages et PFE	Module soutenance limité, coût élevé, complexité d'intégration
Koha / Aleph	Systèmes bibliothèques universitaires	Hors sujet

TABLE 4 – Analyse des systèmes ERP académiques

L'analyse des systèmes ERP académiques révèle une inadéquation fondamentale entre ces solutions et notre besoin. Apogée, bien que largement déployé dans les universités françaises, se focalise sur la gestion administrative des étudiants (inscriptions, notes, diplômes) sans offrir de module dédié à l'organisation des soutenances. Aurion, plus complet, intègre un suivi des stages et des PFE mais son module de planification de soutenances reste basique, sans détection automatique de conflits ni interface de planification interactive. De plus, le coût de licence et la complexité d'intégration de ces solutions les rendent inadaptées à un contexte de faculté marocaine cherchant une solution légère et ciblée.

2.3.2 Outils de planification générique

Solution	Description	Limites pour ce besoin
Doodle	Sondages de disponibilité	Pas de gestion des jurys ni de génération PDF
Google Calendar	Calendrier partagé	Pas de logique métier académique, pas de convocations
Microsoft Teams	Gestion de réunions	Non adapté à la logique jury/soutenance

TABLE 5 – Analyse des outils de planification génériques

Les outils de planification génériques, bien que faciles à prendre en main, souffrent du même défaut : ils ne connaissent pas la logique métier académique. Doodle permet de trouver un créneau commun pour une réunion, mais ne sait pas qu'un enseignant ne peut pas être dans deux salles en même temps, ni qu'un jury doit respecter un minimum de membres. Google Calendar offre une visualisation calendaire mais aucune règle de gestion. Microsoft Teams, conçu pour

la collaboration en entreprise, n'a aucune notion de jury, de salle ou de session de soutenance. Ces outils pourraient éventuellement compléter une solution dédiée, mais ne peuvent en aucun cas la remplacer.

2.3.3 Solutions open source (GitHub)

Projet	Lien GitHub	Limites identifiées
appliSoutenance	cdegrel/appliSoutenance	Focalisé sur la notation en séance ; ne gère pas la planification amont ni les salles
PFE-Management	khaireddine24/PFE-MS	Gestion administrative de base ; absence de détection de conflits et de génération PDF
Jury (HackUTD)	hackutd/jury	Adapté aux hackathons ; incompatible avec les rôles académiques (Président, Rapporteur)

TABLE 6 – Analyse de projets open source similaires

Les projets open source trouvés sur GitHub présentent un intérêt pédagogique mais ne répondent pas à l'ensemble de nos besoins. appliSoutenance se concentre sur la phase de notation pendant la soutenance, sans couvrir la planification amont ni la gestion des salles. PFE-Management offre une gestion administrative de base des projets mais ne détecte pas les conflits d'horaires et ne génère pas de documents. Jury, conçu pour les hackathons, utilise un modèle de jury simplifié incompatible avec les rôles académiques français (Président, Rapporteur, Examineur). Ces projets confirment que la communauté open source n'a pas encore produit de solution complète pour l'organisation des soutenances universitaires.

2.3.4 Synthèse et positionnement

Le tableau 7 résume la couverture de chaque solution par rapport aux quatre critères essentiels identifiés.

Critère	Apogée	Doodle	PFE-MS	Notre système
Cycle complet (planif. → éval.)	Partiel	Non	Non	✓
Gestion jurys + détection conflits	Non	Non	Non	✓
Génération documents A4	Non	Non	Non	✓
Contexte marocain (CNE, filières)	Non	Non	Partiel	✓

TABLE 7 – Synthèse comparative des solutions

Comme le montre ce tableau, aucune solution existante ne réunit simultanément les quatre critères suivants : gestion complète du cycle de soutenance, composition et affectation de jurys avec détection de conflits, génération automatique de convocations et fiches d'évaluation, et adaptation au contexte marocain. Cette analyse confirme la pertinence du projet et justifie le développement d'une solution sur mesure.

2.4 Choix technologiques

Le choix des technologies est une étape cruciale qui conditionne la réussite du projet. Chaque décision a été prise en fonction des besoins spécifiques du système, des compétences de l'équipe et des contraintes du projet. Pour chaque choix, nous avons évalué les alternatives disponibles avant de justifier la sélection retenue.

2.4.1 Frontend

Trois frameworks JavaScript principaux ont été évalués pour le développement de l'interface utilisateur : React.js, Vue.js et Angular. Svelte a également été considéré mais écarté en raison de son écosystème encore trop immature pour un projet de cette envergure.

Vue.js offre une courbe d'apprentissage plus douce que React et une documentation excellentement structurée. Son système de réactivité basé sur les proxies est intuitif et son official store (Pinia) est léger et efficace. Cependant, son écosystème de composants tiers est moins riche que celui de React, et la taille de sa communauté – bien que croissante – reste significativement plus petite, ce qui limite la disponibilité de solutions prêtes à l'emploi pour des cas d'usage spécifiques comme le glisser-déposer complexe.

Angular propose une solution complète et opinionnée, avec un CLI puissant, un système de dépendances injection natif et une architecture en modules. Sa robustesse est éprouvée dans les grandes applications d'entreprise. Néanmoins, sa courbe d'apprentissage élevée, sa verbosité et la taille du bundle produit le rendent moins adapté à un projet de fin d'études où la vélocité de développement est un facteur déterminant.

React.js (v19) a été retenu pour plusieurs raisons complémentaires. Son modèle de composants fonctionnels avec hooks offre une flexibilité sans équivalent, permettant de créer des composants réutilisables et testables. Le DOM virtuel assure des performances optimales même avec des mises à jour fréquentes, ce qui est essentiel pour le Defense Designer où l'état du planning change constamment. L'écosystème React est le plus riche du marché : TanStack Query pour la gestion de l'état serveur, @dnd-kit pour le glisser-déposer, React Router pour le routage, et shadcn/ui pour le design system. Enfin, la version 19 apporte des améliorations significatives en termes de performances et de gestion de la mémoire, ce qui est pertinent pour une application manipulant de grandes quantités de données (plannings, listes d'étudiants, jurys).

L'interface est construite avec **Tailwind CSS 4** et **shadcn/ui**. Tailwind a été préféré à Bootstrap ou Material UI car son approche utility-first permet un contrôle total sur le rendu visuel sans être contraint par les styles prédéfinis d'un framework. shadcn/ui, basé sur Radix UI, fournit des composants d'accessibilité (dialogues, listes déroulantes, menus) entièrement personnalisables via des classes Tailwind, contrairement à Material UI dont les composants imposent une charte graphique difficile à adapter à un design sur mesure.

2.4.2 Backend

L'évaluation du backend a porté sur trois alternatives majeures : Node.js avec Express, Python avec Django, et Java avec Spring Boot.

Node.js/Express permettrait d'utiliser un seul langage (JavaScript) sur toute la pile technique, facilitant ainsi la maintenance et la mobilité de l'équipe entre le frontend et le backend. Cependant, l'absence de typage statique natif (TypeScript atténue partiellement ce problème) et la nature single-threaded de l'événement loop posent des questions de fiabilité pour un système

devant garantir l'intégrité des données de planification. De plus, l'écosystème npm, bien que vaste, souffre de problèmes de dépendances obsolètes et de fragilité des versions.

Django offre un ORM puissant, une administration automatique et une sécurité intégrée. Sa philosophie « batteries included » accélère le développement initial. Néanmoins, l'écosystème Python pour les applications web d'entreprise est moins mature que celui de Java, et la performance d'exécution est inférieure pour les charges de travail intensives en calcul (détection de conflits, optimisation de planning).

Spring Boot 3.4.6 (Java 17) a été choisi pour sa robustesse éprouvée dans les applications d'entreprise. Spring Security fournit un mécanisme d'authentification JWT natif avec un contrôle d'accès granulaire par rôles, sans nécessiter de bibliothèques tierces. Spring Data JPA simplifie considérablement l'accès aux données relationnelles tout en offrant la flexibilité des requêtes JPQL personnalisées. L'architecture MVC contraint les développeurs à une séparation claire des préoccupations (Contrôleurs, Services, Repositories), ce qui facilite la maintenabilité et la testabilité du code. Enfin, Java 17, version LTS (Long Term Support), garantit la stabilité à long terme du projet sans nécessiter de migration de version.

2.4.3 Base de données

La stratégie de base de données adopte une approche à double profil, adaptée aux différentes phases du cycle de développement.

En phase de développement et de test, **H2 Database** est utilisé en mode mémoire. Cette base de données embarquée ne nécessite aucune installation, démarre instantanément et permet de réinitialiser l'état à chaque redémarrage. Le seed data peuple automatiquement la base avec des données de démonstration (100 étudiants, 12 enseignants, 25 projets, 15 soutenances, etc.), ce qui facilite les tests fonctionnels et le développement.

En phase de production, **MySQL** est le SGBD retenu. Son choix se justifie par sa maturité, ses performances éprouvées pour les charges de travail académiques, et sa compatibilité native avec Spring Data JPA et Hibernate. Le schéma de la base est identique entre H2 et MySQL grâce à l'utilisation du sous-ensemble commun du langage SQL et des annotations JPA standardisées. La configuration des profils Spring (`application.properties` pour le dev, `application-prod.properties` pour la production) permet de basculer d'un SGBD à l'autre sans modification du code application.

2.4.4 Outils de développement

Git et GitHub constituent le socle de la gestion de version et de la collaboration. Le projet est organisé en dépôts multiples (API, UI, E2E, Docs), chacun doté de son propre historique de commits et de sa configuration CI/CD. La stratégie de branching suit un modèle simplifié : une branche main stable, des branches de fonctionnalité (`feature/*`) pour le développement, et des branches de correction (`fix/*`) pour les correctifs. Les messages de commit respectent les conventions (préfixes `feat:`, `fix:`, `chore:`, etc.) et les issues GitHub servent de support au suivi des tâches et des bugs.

Visual Studio Code est l'éditeur principal pour le frontend, avec des extensions dédiées au TypeScript, à Tailwind CSS et à React. **IntelliJ IDEA Ultimate** est utilisé pour le backend Java, offrant un support natif de Spring Boot, Maven et JPA. **Postman** permet de tester et documenter les endpoints REST, avec des environnements séparés pour le dev et les tests, et des collections

partagées entre les membres de l'équipe. **Maven** gère les dépendances du projet backend, assure la cohérence des versions et intègre les plugins de qualité de code.

Docker Compose orchestre l'environnement de développement en conteneurisant l'API Spring Boot et le serveur Mailpit (serveur SMTP de test avec interface web sur le port 8025). Cette approche garantit un environnement reproductible sur toute machine, éliminant les problèmes de « ça fonctionne sur ma machine » et simplifiant la prise en main par un nouveau développeur.

2.4.5 Outils de qualité de code

La qualité du code est assurée par une chaîne d'outils intégrée au cycle de développement, combinant des outils de linting, de formatage et de validation statique.

Côté frontend, **ESLint** analyse le code TypeScript en temps réel pour détecter les erreurs courantes (variables non utilisées, imports manquants, patterns dangereux). **Prettier** assure un formatage uniforme de l'ensemble du code source, éliminant les débats stylistiques au sein de l'équipe. **cspell** vérifie l'orthographe des noms de variables et des commentaires, utile dans un projet où l'interface est en français mais le code en anglais.

Côté backend, **Checkstyle** vérifie la conformité du code aux conventions de codage Java (nommage, indentation, longueur des lignes). **PMD** détecte les problèmes de qualité de code (code mort, duplication, complexité cyclomatique excessive). **SpotBugs** analyse le code bytecode pour identifier les bugs potentiels (null pointer, resource leak, thread safety). Ces trois outils sont intégrés au build Maven et exécutés automatiquement à chaque compilation.

Husky et **lint-staged** implémentent des hooks de pré-commit qui exécutent le linting et le formatage sur les fichiers stageés avant chaque commit. Cette barrière de qualité garantit qu'aucun code non formaté ou contenant des erreurs de linting n'atteint le dépôt. Couplé à **Vitest** et **React Testing Library** pour les tests unitaires frontend, et à **JaCoCo** pour la couverture de code Java, cet ensemble d'outils constitue un filet de sécurité complet qui assure la maintenabilité du code à long terme.

2.5 Architecture générale retenue

L'application adopte une architecture web moderne en trois couches, favorisant la modularité et la séparation des préoccupations. La 4 présente la vue composants du système, détaillant les modules de chaque couche.

2.5.1 Philosophie API-first

Le système est conçu selon une approche *API-first*, où l'interface utilisateur et le serveur sont développés de manière indépendante autour d'un contrat API REST formalisé. Cette séparation permet au frontend de consommer les endpoints sans connaître les détails d'implémentation du backend, et inversement. Les endpoints sont documentés via Swagger/OpenAPI (/swagger-ui/index.html), ce qui facilite les tests manuels via Postman et la consommation de l'API par d'autres clients potentiels.

Chaque endpoint suit les conventions REST : utilisation des verbes HTTP appropriés (GET pour la lecture, POST pour la création, PUT pour la mise à jour, DELETE pour la suppression), réponses au format JSON enveloppé dans un objet `ApiResponse<T>` contenant les champs `success`, `message`, `data` et `errors`, et codes de statut HTTP conformes au standard (200

pour le succès, 201 pour la création, 400 pour les erreurs de validation, 404 pour les ressources introuvables, 409 pour les conflits).

2.5.2 Organisation multi-dépôts

Le projet est organisé en quatre dépôts Git distincts, chacun Responsable d'un périmètre technique spécifique. Cette séparation, bien que nécessitant une synchronisation via des workflows CI, offre des avantages significatifs en termes de clarté du code, d'indépendance des cycles de build et de responsabilité des contributions.

Le dépôt `system-gestion-soutenance-api` contient le code Java/Spring Boot du serveur. Le dépôt `system-gestion-soutenance-ui` contient l'application React/TypeScript. Le dépôt `system-gestion-soutenance-e2e` contient les tests Playwright et reçoit les code sources synchronisés des deux premiers dépôts pour ses tests d'intégration. Le dépôt `system-gestion-soutenance-docs` contient la documentation du projet, les diagrammes PlantUML et le rapport LaTeX.

Des workflows GitHub Actions assurent la synchronisation automatique : lors d'un push sur la branche `main` de l'API ou de l'UI, un script `rsync` copie les sources dans le dépôt E2E et pousse les modifications. Cette architecture garantit que les tests d'intégration s'exécutent toujours sur la dernière version du code, tout en maintenant une séparation claire des responsabilités.

2.5.3 Stratégie de déploiement

En environnement de développement, **Docker Compose** orchestre l'ensemble des services nécessaires. Le fichier `docker-compose.yml` définit deux conteneurs : l'API Spring Boot (exposée sur le port 8080) et Mailpit (serveur SMTP de test avec interface web sur le port 8025). L'application frontend est lancée séparément via `npm run dev` (Vite dev server sur le port 5173), avec un proxy configuré pour rediriger les appels `/api` vers le backend.

Cette architecture de développement est conçue pour être lancée en une seule commande (`docker compose up -build` pour le backend, `npm run dev` pour le frontend), offrant ainsi un environnement de travail immédiat pour les développeurs. La base de données H2 en mémoire est automatiquement peuplée par le `DataInitializer` lors du premier démarrage, fournissant des données de démonstration réalistes.

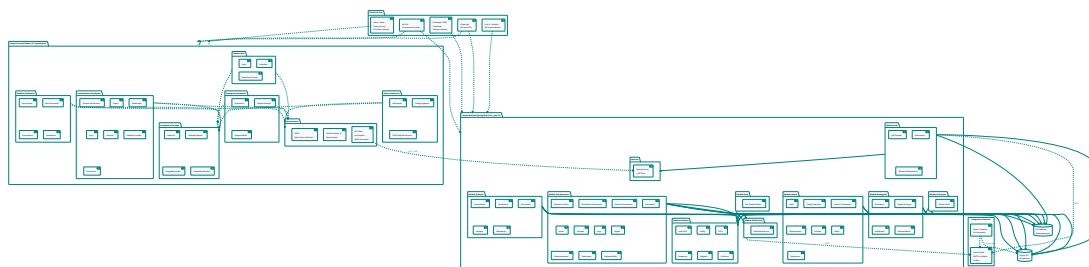


FIGURE 4 – Architecture en composants du système

2.6 Conclusion

Ce chapitre a dressé un état des lieux complet du système actuel d'organisation des soutenances, mettant en évidence ses limites opérationnelles par une analyse quantifiée du temps consacré à chaque étape. L'analyse des solutions existantes – systèmes ERP, outils génériques et projets

open source – a confirmé l'absence d'un outil répondant simultanément à tous les besoins du contexte universitaire marocain. Les choix technologiques – React 19, Spring Boot 3.4.6, MySQL, Docker – ont été justifiés par une évaluation approfondie des alternatives, et la chaîne de qualité de code a été présentée comme garantie de maintenabilité. L'architecture en composants, l'approche API-first et l'organisation multi-dépôts constituent les fondations techniques sur lesquelles repose le système. Le chapitre suivant détaillera l'analyse des besoins fonctionnels et la conception complète du système.

3 Analyse des besoins et conception du système

3.1 Introduction

Ce chapitre détaille la phase de conception du système de gestion des soutenances. Il définit les exigences fonctionnelles et non fonctionnelles, formalise les règles de gestion qui régissent la planification, et présente la modélisation UML du système. Cette étape est cruciale pour garantir que la solution technique répond précisément aux besoins du département. Dans un projet de développement logiciel, la qualité de la phase de conception détermine directement la maintenabilité et l'évolutivité du système résultant. C'est pourquoi nous avons adopté une démarche rigoureuse combinant analyse des besoins, modélisation UML et définition formelle des règles de gestion, afin de disposer d'un référentiel clair avant de passer à l'implémentation.

3.2 Présentation générale du projet

Le projet consiste en la réalisation d'une application web centralisée permettant l'organisation complète des sessions de soutenances. Contrairement à une simple gestion d'agenda, le système agit comme un moteur de planification intelligent capable d'orchestrer des sessions globales (normale, rattrapage) avec des paramètres configurables incluant la durée des soutenances, les pauses entre les passages, et les coefficients d'évaluation par rôle.

Le système couvre l'ensemble du cycle de vie d'une soutenance : de la création des projets et la constitution des jurys, en passant par la planification interactive des créneaux via un éditeur visuel par glisser-déposer (*Defense Designer*), jusqu'à la saisie des évaluations et la génération automatique des documents administratifs (convocations, procès-verbaux, fiches d'évaluation). Chaque étape est validée par un moteur de détection de conflits garantissant l'intégrité du planning.

Le périmètre du projet a été défini en collaboration avec l'encadrant et le département d'accueil. Sont inclus dans le périmètre : la gestion des utilisateurs et des rôles, la configuration académique (filières, niveaux, salles), la gestion des projets et des groupes d'étudiants, la constitution des jurys, la planification interactive des créneaux, la détection automatique de conflits, la saisie des évaluations, la génération de documents et le système de notifications. Sont exclus du périmètre : la gestion financière, la publication des résultats finaux (bulletins), l'intégration avec le système d'information universitaire existant, et le développement d'une application mobile.

3.3 Cahier des charges

3.3.1 Acteurs du système

Le système repose sur une gestion stricte des accès basée sur les rôles (*Role-Based Access Control* — *RBAC*). Quatre acteurs principaux sont identifiés, chacun disposant d'un périmètre fonctionnel spécifique. Le choix de quatre rôles uniquement, sans possibilité de superposition, simplifie considérablement la logique de contrôle d'accès tout en couvrant l'ensemble des besoins identifiés.

L'**Administrateur** est responsable de la configuration structurelle de l'établissement. Il gère les départements, filières, niveaux et salles. Il est également en charge de la création des comptes utilisateurs (import CSV, activation) et de la configuration globale des paramètres du système

(durée des soutenances, templates de jurys, configuration email). Ce rôle est le seul à pouvoir accéder à la configuration système et aux journaux d'audit.

Le **Coordinateur** est l'acteur central du système. Il gère l'ensemble du cycle de planification : import et validation des projets, constitution des jurys selon les templates de rôles, utilisation du *Defense Designer* pour la planification interactive des créneaux, consultation du tableau de bord des conflits, et suivi des évaluations. Il peut également publier le planning final et générer les documents administratifs. Ce rôle nécessite une vue d'ensemble de toutes les soutenances et une compréhension des contraintes de chaque acteur.

L'**Enseignant** est membre du corps professoral impliqué dans les soutenances. Il consulte son planning personnel, déclare ses indisponibilités via un calendrier interactif, saisit les notes des étudiants qui lui sont assignés, et accède à ses convocations. Son interaction avec le système est principalement consultative, à l'exception de la saisie des notes et de la déclaration d'indisponibilités.

L'**Étudiant** est l'utilisateur final du système. Il visualise les informations relatives à sa soutenance (date, salle, jury), consulte la composition de son groupe de projet, et télécharge sa convocation officielle au format A4. Son périmètre d'action est le plus limité, ce qui est cohérent avec son rôle dans le processus de soutenance.

Le tableau 8 résume la répartition des fonctionnalités principales par acteur.

Fonctionnalité	Admin	Coord.	Enseign.	Étudiant
Gestion départements	✓	—	—	—
Gestion salles	✓	—	—	—
Gestion utilisateurs	✓	—	—	—
Configuration système	✓	—	—	—
Gestion projets	—	✓	—	—
Constitution jurys	—	✓	—	—
Planification créneaux	—	✓	—	—
Détection conflits	—	✓	—	—
Saisie indisponibilités	—	—	✓	—
Saisie évaluations	—	—	✓	—
Consultation planning	—	✓	✓	✓
Consultation groupe	—	—	—	✓
Téléchargement documents	—	—	—	✓

TABLE 8 – Répartition des fonctionnalités par acteur

3.3.2 Besoins fonctionnels

Les besoins sont organisés par modules métier, chacun correspondant à un sous-système cohérent de l'application. Chaque module a été identifié lors des entretiens avec le coordinateur et l'encadrant, puis priorisé en fonction de son impact sur le processus global de soutenance.

Le **module Gestion Identité** assure l'authentification sécurisée via JWT avec tokens d'accès et de rafraîchissement. Il gère la récupération de mot de passe avec lien à durée limitée (1 heure), l'activation de compte avec validation de la force du mot de passe (via la bibliothèque *zxcvbn*), et l'interface de connexion unifiée avec redirection automatique basée sur le rôle de l'utilisateur. Ce module constitue la porte d'entrée du système et doit garantir la confidentialité des données tout en offrant une expérience fluide.

Le **module Gestion Académique** couvre l'administration des filières (MIP, BCG, etc.), des niveaux (L3, M1, M2) et des grades académiques. Il inclut l'import massif des étudiants et des salles via fichiers CSV avec validation des données et gestion des erreurs ligne par ligne. L'interface CRUD complète avec recherche et pagination permet de maintenir les référentiels à jour tout au long de l'année universitaire.

Le **module Gestion des Projets et Jurys** gère la création et le suivi des projets avec des statuts (proposé, approuvé, rejeté). Il permet la constitution des jurys selon des templates configurables avec rôles normalisés (Président, Rapporteur, Examineur), et assure le suivi de la charge des enseignants pour éviter les sur-affectations. Ce module est le point de départ du processus de planification, car les projets et jurys sont les entités qui seront ensuite placés dans le planning.

Le **module Planification (Defense Designer)** est l'élément le plus innovant du système. Il fournit une interface interactive de type canevas permettant au coordinateur de planifier les soutenances par glisser-déposer. Les jurys sont listés dans une barre latérale et peuvent être déposés sur un calendrier dynamique organisé par salles et créneaux horaires. L'état local est maintenu côté client et synchronisé avec le serveur uniquement lors de la sauvegarde explicite. Un algorithme d'auto-génération permet de peupler automatiquement le planning en optimisant l'occupation des salles.

Le **module Moteur de Conflits** assure la validation systématique de 8 règles de gestion lors de chaque modification du planning (voir 3.4). Le moteur est dupliqué côté client pour un retour instantané lors des interactions et exécuté côté serveur pour garantir l'intégrité lors de la sauvegarde. Cette architecture double-validation est un choix architectural majeur qui combine réactivité et fiabilité.

Le **module Évaluation** gère la saisie des notes par rôle de jury avec des coefficients dynamiques configurés par session. Le calcul automatique de la moyenne pondérée élimine les erreurs de calcul manuel. Le suivi de l'avancement des évaluations avec indication visuelle du statut (en attente, complétée) permet au coordinateur d'identifier rapidement les évaluations en retard.

Le **module Édition Documentaire** génère 5 types de documents (convocation jury, convocation étudiant, fiche d'évaluation, liste de présence, procès-verbal) via des vues React optimisées pour l'impression A4 utilisant la stratégie `@media print`. Cette approche client-side évite la complexité d'une génération PDF serveur tout en produisant des documents de qualité professionnelle.

Le **module Notifications et Audit** fournit un système de notifications internes avec différents niveaux (success, warning, error) et la journalisation des actions critiques avec piste d'audit complète pour la traçabilité. Chaque action significative (création, modification, suppression) est enregistrée avec l'identifiant de l'utilisateur, l'horodatage et les détails de l'opération.

3.3.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les qualités transverses du système, souvent aussi critiques que les fonctionnalités elles-mêmes pour garantir l'adoption et la satisfaction des utilisateurs.

La **réactivité** est un besoin fondamental pour le Defense Designer. Le retour sur conflit lors du drag-and-drop doit être instantané (latence < 100ms) grâce à la validation locale via le moteur de conflits TypeScript. L'interface utilisateur ne doit pas présenter de blocage perceptible lors

des interactions, ce qui impose de maintenir l'état du planning dans la mémoire du navigateur et de ne jamais effectuer d'appels réseau pendant les opérations de glisser-déposer.

La **sécurité** repose sur le chiffrement des mots de passe via BCrypt, la protection des endpoints par tokens JWT avec validation côté serveur (Spring Security), et le contrôle d'accès basé sur les rôles via annotations `@PreAuthorize`. Les tokens ont une durée de validité limitée (2 heures) et sont stockés dans le `localStorage` du navigateur. Une attention particulière a été portée à la protection contre les attaques par injection SQL, les failles XSS et les erreurs d'autorisation horizontale.

La **disponibilité** est assurée par une architecture stateless du backend, permettant un déploiement horizontal scalable. L'API REST ne maintient pas d'état de session côté serveur, chaque requête étant autonome via le token JWT. Cette conception facilite également les tests et le débogage, car chaque requête contient toute l'information nécessaire à son traitement.

L'**ergonomie** vise une interface moderne et responsive basée sur Tailwind CSS 4 et shadcn/ui. La navigation est contextualisée par rôle avec une sidebar adaptative qui ne présente que les fonctionnalités accessibles à l'utilisateur connecté. Le thème clair/sombre avec persistance du choix améliore le confort d'utilisation lors de sessions de travail prolongées.

La **maintenabilité** est garantie par une architecture en couches stricte (Contrôleur → Service → Repository → Entité) avec séparation claire des responsabilités. L'utilisation de DTOs pour l'isolation des couches API et de MapStruct pour le mapping objet protège le modèle interne des variations de l'interface exposée.

La **qualité** est mesurée par une couverture de code minimale de 85% (lignes) et 70% (branches) via JaCoCo côté Java. Le linting automatisé avec ESLint (TypeScript) et Spotless/Checkstyle/PMD/SpotBugs (Java) intégré dans le cycle Maven assure la conformité du code aux conventions de l'équipe.

3.4 Règles de gestion

Le moteur de conflits implémente les règles de gestion (RG) suivantes, garantissant l'intégrité du planning. Ces règles ont été définies en collaboration avec le coordinateur du département MIP et formalisées sous forme de contraintes vérifiables algorithmiquement.

Règles de Gestion du Planning:

RG1 (Enseignants) Un enseignant ne peut être affecté à deux soutenances dont les horaires se chevauchent.

RG2 (Salles) Une salle ne peut accueillir qu'une seule soutenance pour un créneau donné.

RG3 (Capacité) Le nombre d'étudiants d'un projet ne doit pas excéder la capacité de la salle assignée.

RG4 (Période) Toutes les soutenances d'une session doivent être planifiées entre la *startDate* et la *endDate* de ladite session.

RG5 (Encadrants) Un encadrant ne doit pas être sollicité sur plusieurs projets le même jour pour éviter la surcharge.

RG6 (Unicité) Un projet ne peut être planifié qu'une seule fois par session.

RG7 (Pause) Un intervalle minimum (*breakDuration*) doit être respecté entre deux soutenances dans une même salle.

RG8 (Disponibilité) Aucune soutenance ne peut être planifiée sur un créneau déclaré comme indisponible par un membre du jury.

Ces règles sont implémentées dans deux moteurs distincts mais fonctionnellement identiques : un moteur Java côté serveur (`ConflictDetectionService`) qui valide lors de la sauvegarde, et un moteur TypeScript côté client (`conflict-engine.ts`) qui fournit un retour instantané lors des interactions dans le Defense Designer.

Le tableau 9 associe chaque règle à son ou ses implémentations et à la sévérité de la violation. Les règles classées « error » bloquent l'opération, tandis que les règles classées « warning » affichent une alerte mais autorisent l'enregistrement.

Règle	Description	Java	TypeScript	Sévérité
RG1	Chevauchement enseignant	✓	✓	error
RG2	Occupation de salle	✓	✓	error
RG3	Capacité de salle	✓	✓	error
RG4	Bornes de période	✓	✓	error
RG5	Surcharge encadrant	✓	✓	warning
RG6	Unicité de projet	✓	✓	error
RG7	Intervalle de pause	✓	✓	error
RG8	Indisponibilité	✓	✓	error

TABLE 9 – Correspondance entre les règles de gestion et leurs implémentations

Lorsqu'une règle ne peut être satisfaite (par exemple, aucun créneau disponible pour un jury donné), le système ne propose pas de mécanisme d'override automatique. Le coordinateur doit soit modifier les paramètres de la session (dates, salles), soit reconfigurer la composition du jury, soit déclarer une indisponibilité compensatoire. Cette approche conservatrice garantit l'intégrité du planning au prix d'une flexibilité réduite, un choix délibéré compte tenu du caractère critique des contraintes académiques.

3.5 Conception UML

La modélisation UML permet de formaliser les différentes visions du système : qui fait quoi (cas d'utilisation), comment les données sont structurées (classes), comment les composants interagissent (séquence), et comment les processus se déroulent (activité). Nous avons utilisé les diagrammes UML 2.5, implémentés via PlantUML pour faciliter la maintenance et la régénération automatique des figures.

3.5.1 Diagrammes de cas d'utilisation

Les diagrammes de cas d'utilisation délimitent les responsabilités de chaque acteur. Ils ont été construits à partir des entretiens avec le coordinateur et l'encadrant, puis validés lors des revues de conception.

Le diagramme de la 5 présente la hiérarchie générale des acteurs. L'acteur de base « Utilisateur » est spécialisé en quatre rôles distincts, chacun héritant de la capacité de s'authentifier. Cette

hiérarchie simple reflète l'architecture RBAC du système, où chaque rôle correspond à un ensemble disjoint de permissions.

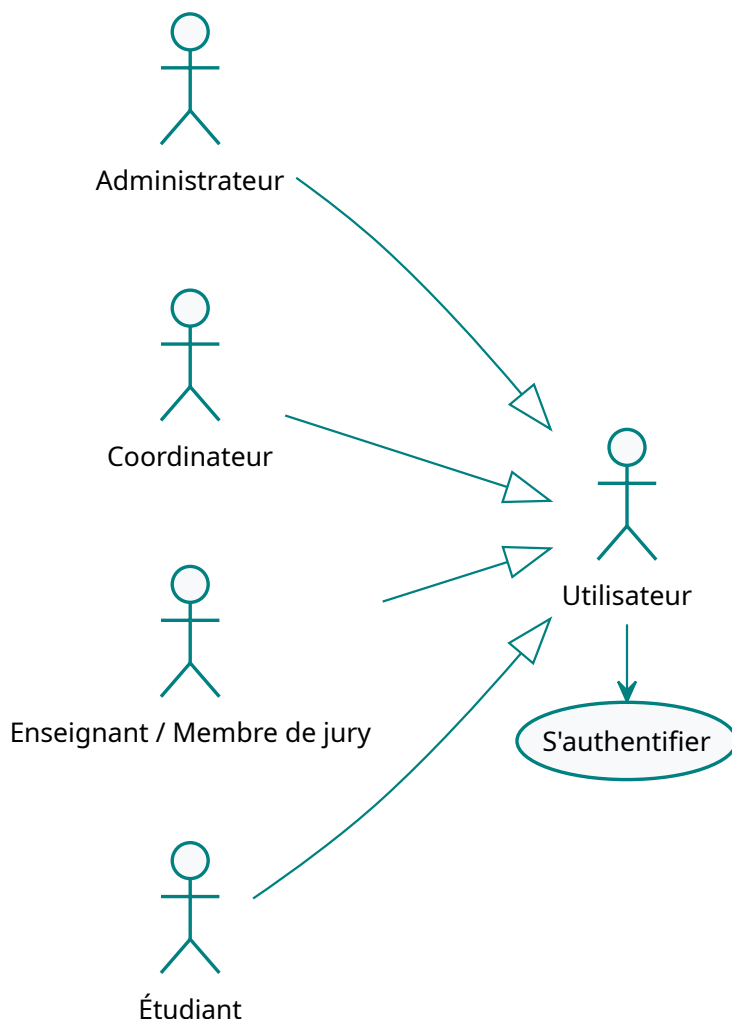


FIGURE 5 – Hiérarchie générale des acteurs

Le diagramme de la 6 détaille les cas d'utilisation de l'Administrateur. Ce rôle est le seul à pouvoir configurer les structures académiques (départements, filières, salles), gérer les comptes utilisateurs (création, modification, suppression, import CSV) et consulter les journaux d'audit. Il configure également les paramètres globaux du système (templates de jurys, durée des soutenances, configuration email).

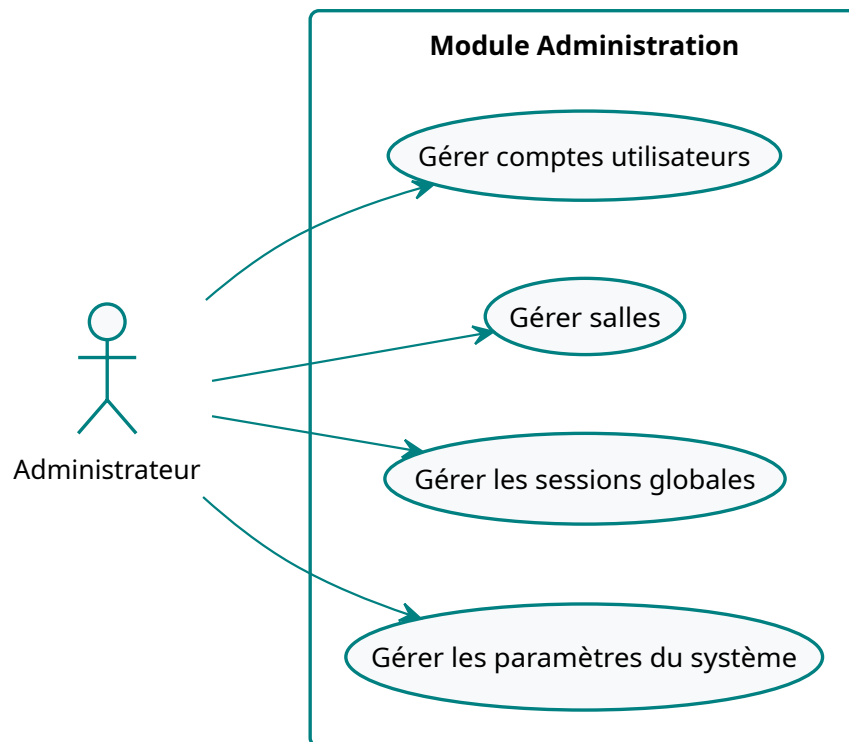


FIGURE 6 – Cas d'utilisation : Administration du système

Le diagramme de la 7 détaille les cas d'utilisation du Coordinateur, l'acteur le plus sollicité par le système. Il gère le cycle complet : des projets et groupes d'étudiants, à la constitution des jurys, en passant par la planification interactive via le Defense Designer, la validation du planning, et la génération des documents. Les relations « include » et « extend » modélisent les dépendances entre cas d'utilisation (par exemple, « Affecter un jury » inclut « Vérifier les conflits »).

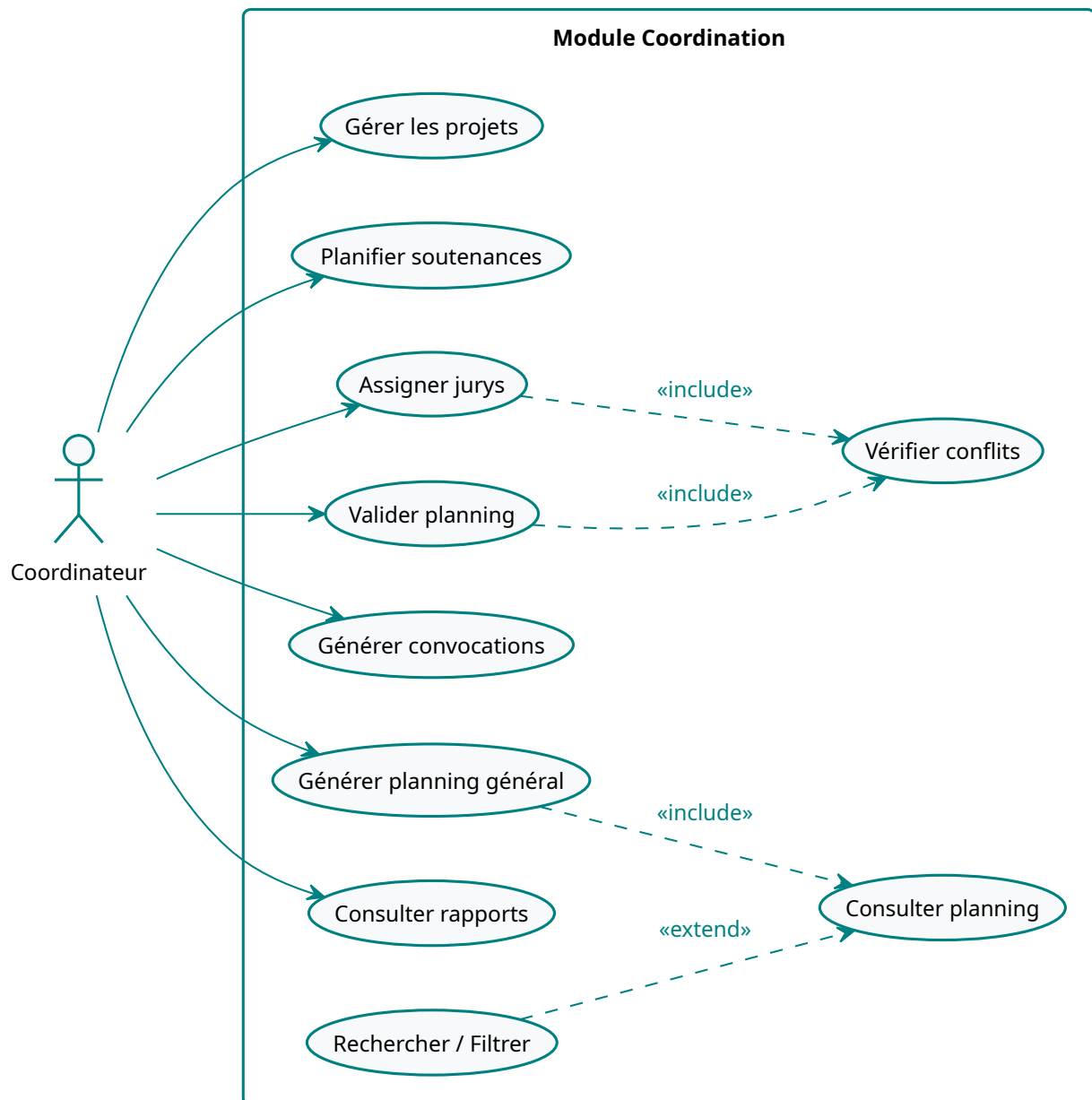


FIGURE 7 – Cas d'utilisation : Coordination et Defense Designer

Le diagramme de la 8 présente les cas d'utilisation de l'Enseignant en tant que membre de jury. Il consulte son planning personnel, déclare ses indisponibilités via un calendrier interactif, saisit les notes et observations pour les étudiants qui lui sont assignés, et accède à ses convocations. Le cas « Saisir les évaluations » inclut la consultation des informations de l'étudiant, garantissant que le jury dispose du contexte nécessaire avant de noter.

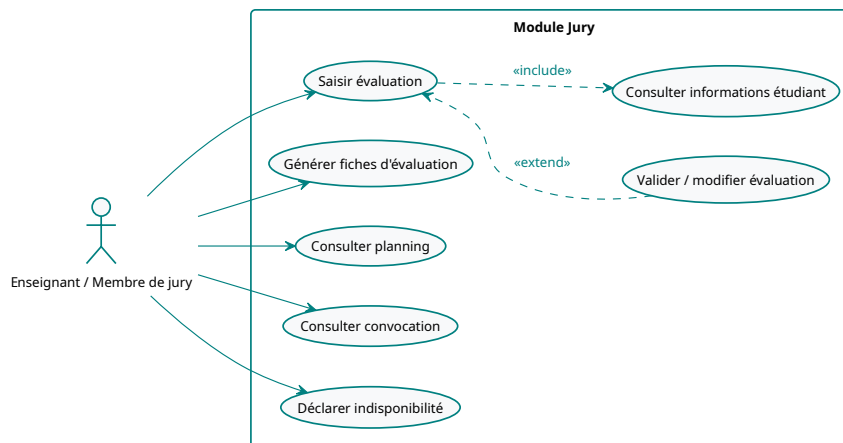


FIGURE 8 – Cas d'utilisation : Enseignant / Membre de jury

Le diagramme de la 9 présente les cas d'utilisation de l'Étudiant. Son périmètre est le plus restreint : il consulte les informations relatives à sa soutenance (date, salle, jury), télécharge sa convocation, et gère son groupe de projet (création, recherche de coéquipiers). Le cas « Télécharger la convocation » inclut la consultation des informations de soutenance, car la convocation doit contenir la date, l'heure, la salle et la composition du jury.

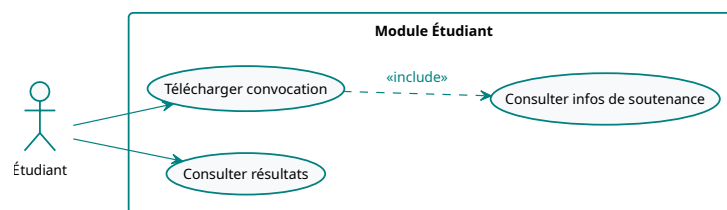


FIGURE 9 – Cas d'utilisation : Étudiant

3.5.2 Diagramme de classes

Le diagramme de classes traduit la structure des données persistées. On y observe une hiérarchie d'utilisateurs basée sur l'héritage JOINED (User avec spécialisation par rôle via le discriminant dtype), et les relations centrales entre les entités métier.

Plusieurs choix de conception méritent d'être soulignés. La classe JuryMember est une entité @Embeddable stockée dans une table d'association defense_members, plutôt qu'une entité JPA à part entière. Ce choix simplifie la gestion des jurys en évitant une table supplémentaire tout en permettant de stocker le rôle du membre (Président, Rapporteur, Examineur) et la référence vers l'enseignant. La classe Evaluation stocke le teacherId sous forme de Long plutôt que d'une relation @ManyToOne, ce qui permet de conserver les évaluations même en cas de modification de la composition du jury. La table d'association defense_session_coefficients (@ElementCollection) permet de pondérer dynamiquement les notes selon le rôle du membre du jury au sein de chaque session, offrant ainsi une flexibilité maximale dans la configuration de la notation.

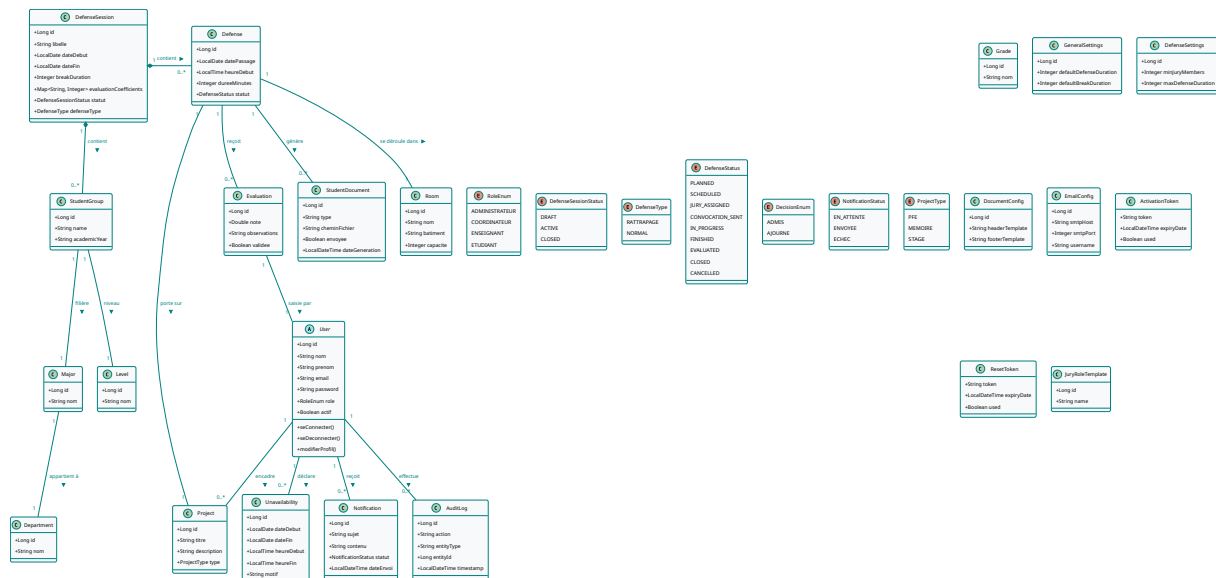


FIGURE 10 – Diagramme de classes global du système

3.5.3 Diagrammes de séquence

Les diagrammes de séquence modélisent les interactions entre les composants du système pour des cas d'utilisation critiques. Nous en présentons trois qui illustrent les flux les plus complexes.

Le diagramme de la 11 modélise le flux de planification interactive via le Defense Designer. Ce processus repose sur un cycle de validation itératif où le frontend et le backend collaborent pour garantir l'absence de conflits :

1. L'utilisateur fait glisser un jury depuis la barre latérale vers une cellule du calendrier.
2. Le hook `useDefenseSchedule` construit le contexte (`ConflictContext`) à partir des données locales.
3. Le moteur TypeScript (`validateSlotAssignment`) exécute les 8 vérifications.
4. En cas d'erreur, un toast est affiché avec une suggestion de résolution ; le dépôt est annulé.
5. En cas de succès, l'état local (`schedule`) est mis à jour immédiatement (*optimistic update*).
6. Lors de la sauvegarde, les données sont envoyées au serveur qui ré-exécute les vérifications via `ConflictDetectionService` avant persistance.

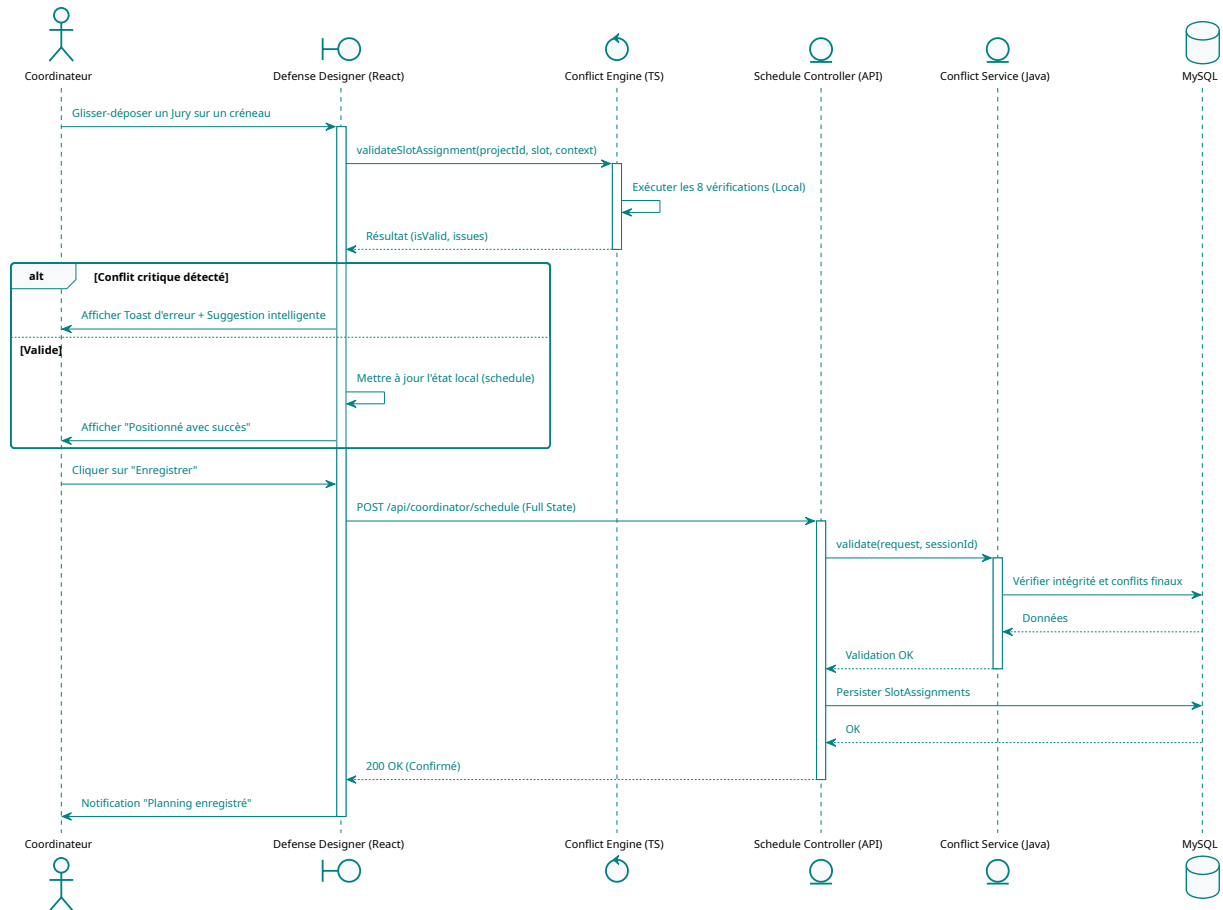


FIGURE 11 – Séquence de planification interactive via le Defense Designer

Le diagramme de la 12 modélise le flux d'authentification. L'utilisateur soumet ses identifiants via le formulaire de connexion, le frontend envoie une requête POST à `/api/auth/login`, le serveur valide les credentials (vérification du hash BCrypt et du statut du compte), puis retourne un token JWT contenant l'identifiant utilisateur et son rôle. Le frontend stocke le token dans le `localStorage` et redirige l'utilisateur vers la page correspondant à son rôle.

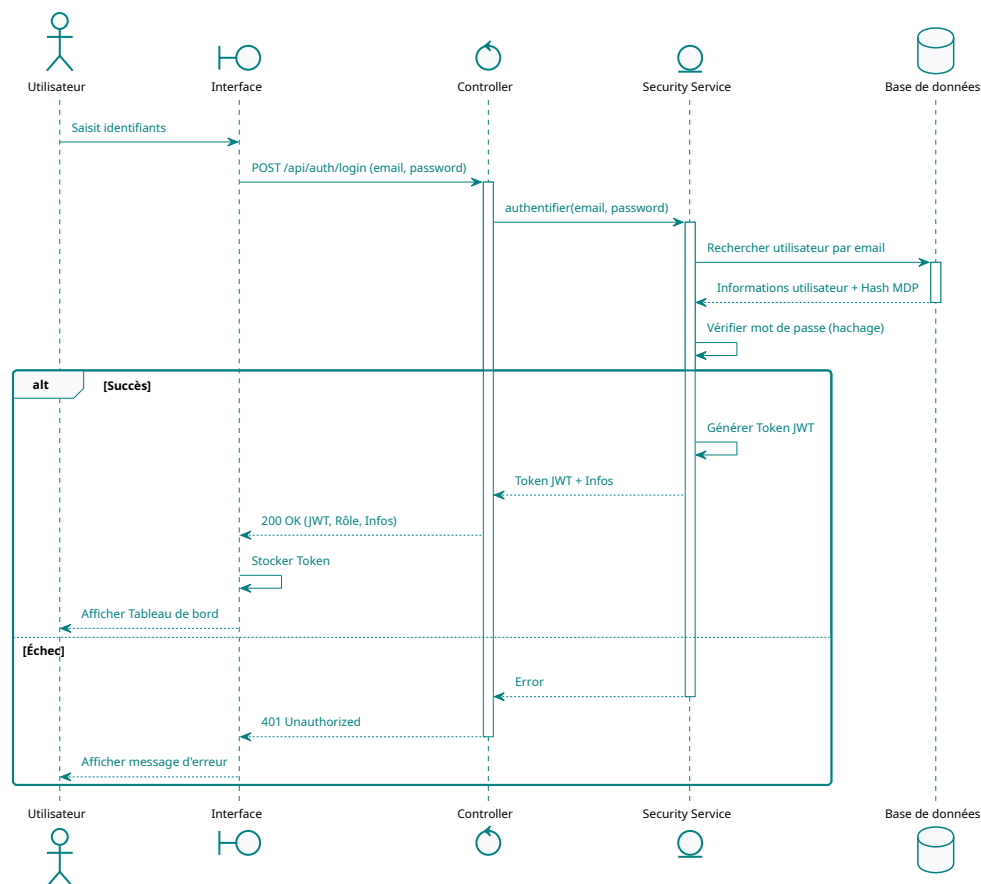


FIGURE 12 – Séquence d'authentification via JWT

Le diagramme de la 13 modélise le flux de soumission d'une évaluation. L'enseignant saisit le score et le commentaire pour un étudiant, le frontend envoie une requête POST à l'endpoint d'évaluation, le serveur valide la note (plage 0–20), enregistre l'évaluation, met à jour le statut (PENDING → SUBMITTED), et retourne la confirmation. Le moteur de notation calculera ensuite la moyenne pondérée une fois toutes les évaluations d'un jury soumises.

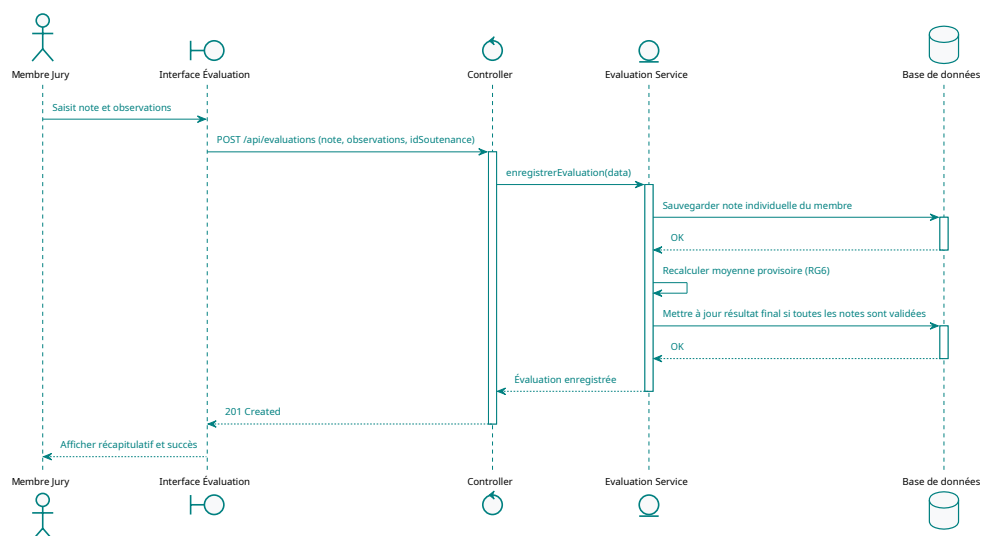


FIGURE 13 – Séquence de soumission d'une évaluation

3.5.4 Diagrammes d'activité et de machine d'états

L'algorithme de détection des conflits, moteur central du système, est modélisé par le diagramme d'activité de la 14. Chaque vérification est une étape indépendante, permettant d'identifier l'ensemble des violations en un seul passage. L'algorithme exécute les 8 règles de gestion en séquence et collecte toutes les violations détectées avant de retourner le résultat au frontend.



FIGURE 14 – Logique de validation des 8 règles de gestion (RG1-RG8)

Le diagramme de la 15 modélise le processus d'évaluation d'une soutenance, de la saisie des notes par les membres du jury jusqu'à la clôture de la session. Les notes sont saisies individuellement par chaque membre, puis le système calcule automatiquement la moyenne pondérée en appliquant les coefficients configurés pour la session. Une fois toutes les évaluations soumises, une décision est proposée (Admis, Admis avec corrections, Ajouré) et le procès-verbal peut être généré.

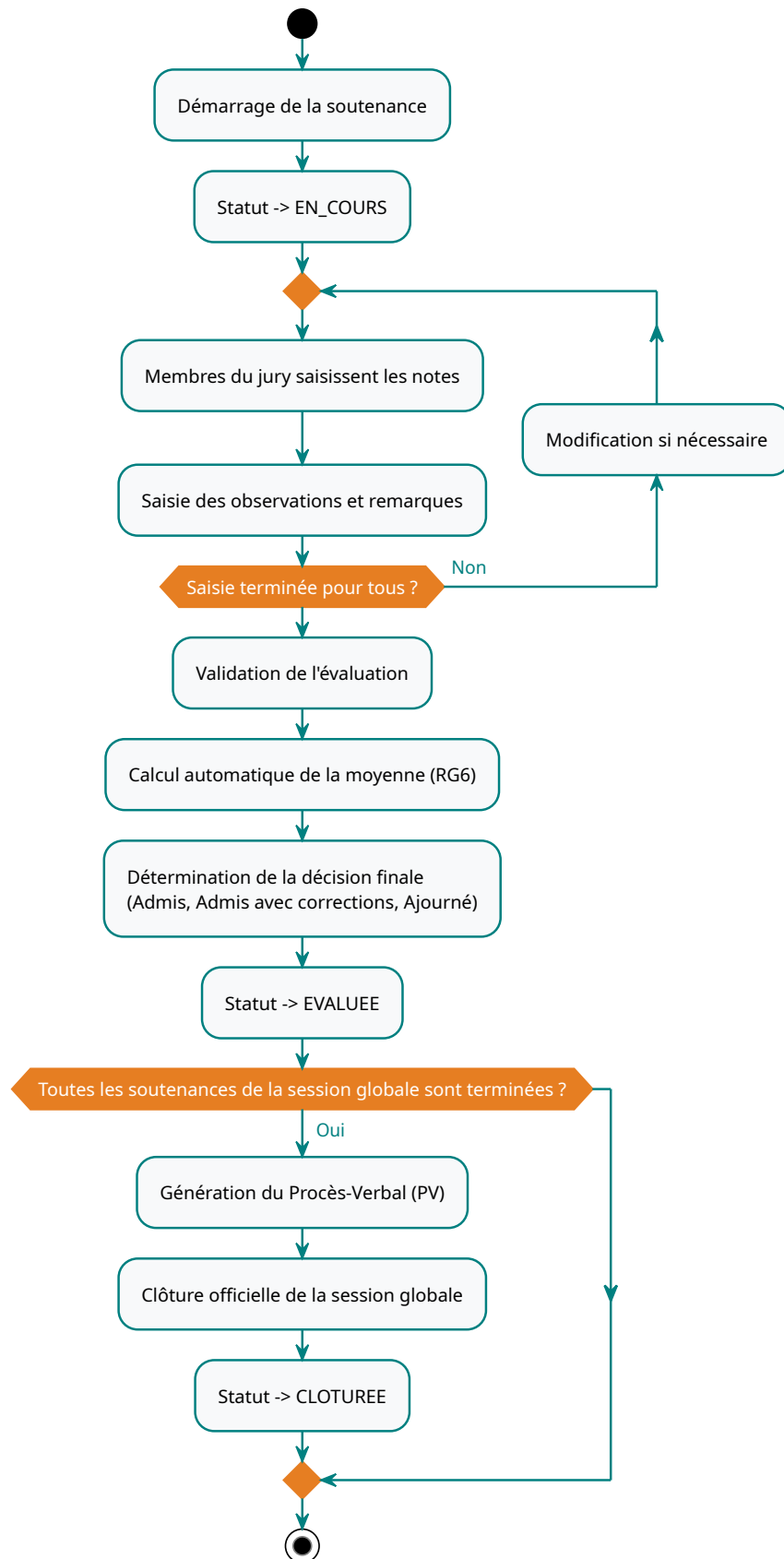


FIGURE 15 – Processus d'évaluation et de notation d'une soutenance

Le diagramme de la 16 modélise le cycle de vie complet d'une soutenance via une machine à

états. La soutenance traverse successivement les états : CREEE, PLANIFIEE, JURY_AFFECTE, CONVOQUEE, EN_COURS, EVALUEE, et CLOTUREE. L'état ANNULE reachable depuis PLANIFIEE, CONVOQUEE et EVALUEE permet de gérer les annulations à différents stades du processus. Cette modélisation garantit que chaque soutenance suit un parcours validé et que les transitions non autorisées sont bloquées par le système.

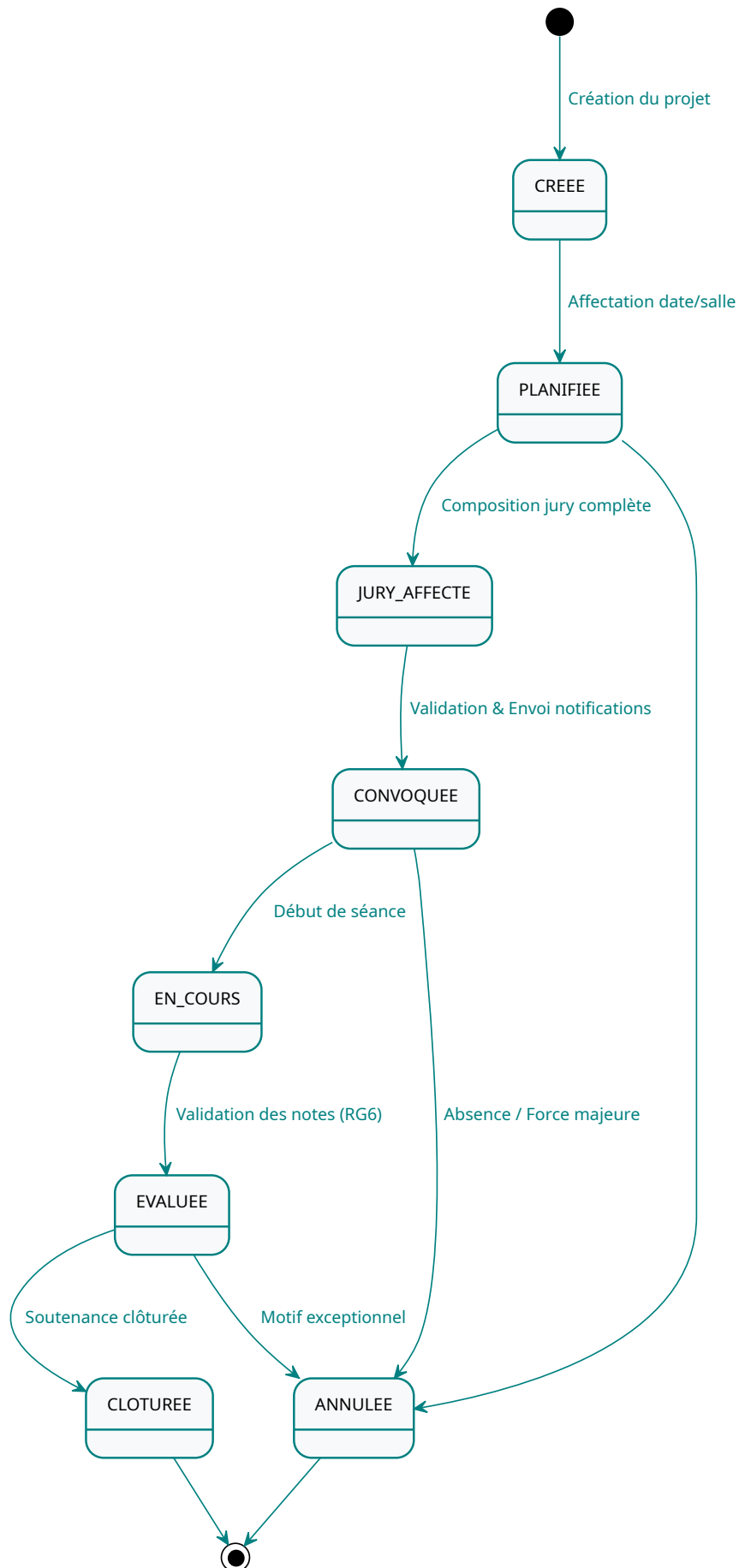


FIGURE 16 – Machine à états du cycle de vie d'une soutenance

3.6 Architecture logicielle

Le système adopte une architecture découplée pour maximiser la maintenabilité et la testabilité. La séparation frontend/backend via une API REST permet à chaque couche d'évoluer indépendamment.

3.6.1 Backend (API REST)

Le backend suit un pattern en couches strict : **Contrôleur** → **Service** → **Repository** → **Entité**. Cette structure permet d'isoler la logique métier (notamment le `ConflictDetectionService`) de la couche d'accès aux données. Les DTOs assurent le découplage entre les API exposées et le modèle interne. Le projet est organisé en dix packages fonctionnels :

- `auth/` – authentification JWT, mot de passe oublié, activation de compte
- `admin/` – gestion des entités académiques (salles, départements, filières, configuration)
- `coordinator/` – cœur métier (projets, groupes, jurys, planification, conflits, notes, documents)
- `teacher/` – évaluations, indisponibilités, planning personnel
- `student/` – groupe, documents, convocation, statistiques
- `notification/` – système de notifications et email
- `common/` – configuration Spring Security, exceptions, mappers, audit
- `seed/` – initialisation des données de démonstration
- `user/` – hiérarchie des utilisateurs (User, Teacher, Student, Coordinator)

3.6.2 Frontend (SPA)

L'interface est conçue comme une application monopage (*Single Page Application*) utilisant React 19 avec TypeScript 6 en mode strict. L'état asynchrone est géré par **TanStack Query v5**, qui assure la synchronisation avec l'API, la mise en cache et le rechargement automatique. Le *Defense Designer* implémente un état local optimiste pour une fluidité maximale lors du drag-and-drop, grâce à la bibliothèque `@dnd-kit/core`. Chaque page est chargée via `React.lazy()` pour optimiser le temps de chargement initial, et les routes sont protégées par des gardes `ProtectedRoute` qui vérifient le rôle de l'utilisateur avant d'autoriser l'accès.

3.6.3 Sécurité et contrôle d'accès

Le flux de sécurité repose sur trois mécanismes complémentaires. L'authentification utilise des tokens JWT signés avec une clé HMAC-SHA, contenant l'identifiant utilisateur et son rôle. Le filtre `JwtAuthFilter` intercepte chaque requête, extrait le token du header `Authorization` ou du cookie `jwt_token`, le valide, et configure le contexte de sécurité Spring. Le contrôle d'autorisation est implémenté via les annotations `@PreAuthorize` sur les contrôleurs, qui restreignent l'accès aux endpoints en fonction du rôle de l'utilisateur courant. Enfin, la configuration CORS autorise les origines du frontend en développement (`localhost:5173`) et en production.

3.7 Modélisation de la base de données

Le Modèle Logique de Données (MLD) traduit les classes JPA en tables relationnelles. La base de données H2 en développement utilise le même schéma que MySQL en production, garantissant la compatibilité entre les environnements.

3.8 Normes et standards respectés

Le développement du système suit des normes et standards industry pour garantir la qualité, la maintenabilité et la cohérence du code source.

Côté Java, le code respecte les conventions de nommage Java (camelCase pour les méthodes et variables, PascalCase pour les classes), les directives de codage Google (limitation à 100 caractères par ligne, gestion des imports non utilisés), et les bonnes pratiques Spring (injection de dépendances par constructeur, annotations de validation sur les DTOs). Les noms de tables et de colonnes suivent la convention snake_case en base de données, reflétée par les annotations `@Table` et `@Column`.

Côté TypeScript, le code suit les conventions Airbnb (formatage Prettier, règles ESLint strictes), le typage strict est activé (`strict: true` dans le `tsconfig.json`), et les noms de fichiers utilisent le kebab-case (`conflict-engine.ts`, `use-defense-schedule.ts`). Les composants React suivent le pattern fonctionnel avec hooks, sans/classes.

Les messages de commit suivent les conventions Git : préfixes sémantiques (`feat:`, `fix:`, `chore:`, `docs:`), description en anglais, imperative mood. Les issues GitHub documentent les fonctionnalités et les corrections, avec des labels de priorité et de catégorie.

3.9 Conclusion

Ce chapitre a permis de formaliser les besoins du système et de concevoir une architecture capable de supporter la complexité de la planification académique. La définition rigoureuse des 8 règles de gestion, la modélisation UML complète (cas d'utilisation, classes, séquence, activité, machine à états) et la description de l'architecture logicielle en couches servent désormais de base technique pour l'implémentation détaillée qui sera présentée au chapitre suivant. Le respect des normes de codage et des standards industry garantit la maintenabilité du système à long terme.

4 Réalisation et développement

4.1 Introduction

Ce chapitre détaille la mise en œuvre technique du système de gestion des soutenances. Nous présentons l'environnement de développement, l'architecture logicielle détaillée, ainsi que l'implémentation des modules fonctionnels avec des extraits de code représentatifs. Un accent particulier est mis sur le moteur de détection de conflits et l'interface interactive de planification, qui constituent le cœur technique du projet. Ce chapitre représente la partie la plus dense du rapport, car il traduit la conception présentée au chapitre précédent en implémentation concrète, en justifiant chaque choix technique par les contraintes métier identifiées lors de l'analyse des besoins.

4.2 Environnement de développement

4.2.1 Matériel et Logiciels

Le développement a été conduit dans un environnement hybride Windows et Linux, utilisant des outils favorisant la collaboration et la qualité du code. Le tableau 10 résume l'ensemble des outils et versions utilisés tout au long du projet.

Catégorie	Outil	Version
Langage backend	Java	17 (LTS)
Framework backend	Spring Boot	3.4.6
Build backend	Maven	3.9+
Langage frontend	TypeScript	6 (strict)
Framework frontend	React	19
Bundler frontend	Vite	8
CSS framework	Tailwind CSS	4
Composants UI	shadcn/ui	—
State management	TanStack Query	v5
Routing	React Router	v7
Drag & drop	@dnd-kit/core	—
Tests backend	JUnit 5 + Mockito	—
Couverture backend	JaCoCo	—
Tests frontend	Vitest + React Testing Library	—
Tests E2E	Playwright	1.52
Mocking frontend	MSW (Mock Service Worker)	v3
Conteneurisation	Docker Compose	—
Email (dev)	Mailpit	—
API testing	Postman	—
CI/CD	GitHub Actions	—

TABLE 10 – Outils et versions utilisés dans le projet

4.2.2 Workflow de développement

Le projet suit un workflow de développement organisé en cycles courts. Chaque fonctionnalité est développée sur une branche dédiée (`feature/*`), testée localement, puis mergée sur la branche `main` via une pull request. Les hooks de pré-commit (Husky + lint-staged) exécutent automatiquement le linting et le formatage avant chaque commit, garantissant que le code poussé est toujours conforme aux standards de l'équipe.

La stratégie de branching est simplifiée mais efficace : la branche `main` est toujours dans un état stable et déployable, les branches `feature/*` vivent le temps du développement d'une fonctionnalité, et les branches `fix/*` sont utilisées pour les corrections de bugs. Les messages de commit suivent les conventions sémantiques : `feat` : pour les nouvelles fonctionnalités, `fix` : pour les corrections, `chore` : pour la maintenance, `docs` : pour la documentation, et `test` : pour les ajouts de tests.

Le pipeline CI/CD (GitHub Actions) automatise la construction et les tests à chaque push. Trois workflows distincts couvrent les trois composants : l'API est construite avec Maven (`./mvnw package`), l'UI est bundlé avec Vite (`npm run build`), et les tests E2E s'exécutent avec Playwright contre les deux serveurs démarrés en parallèle. La synchronisation entre les dépôts est assurée par des scripts `rsync` qui copient les sources dans le dépôt E2E lors de chaque fusion sur `main`.

4.3 Architecture technique du système

L'application suit une architecture web moderne et découplée (API REST + SPA), assurant une séparation nette entre la logique métier et la présentation.

4.3.1 Le Backend : API REST Spring Boot

Le backend est structuré selon un pattern en couches strict pour garantir la maintenabilité. Chaque couche a une responsabilité clairement définie et communique avec les couches adjacentes via des interfaces bien définies.

La **couche Contrôleur** expose les endpoints REST et gère la validation des entrées via les annotations `@Valid` et les DTOs de requête. Les contrôleurs utilisent l'injection de dépendances pour accéder aux services métier, et retournent des réponses enveloppées dans `ApiResponse<T>` pour assurer une cohérence format. Le paramètre `@AuthenticationPrincipal User user` permet d'extraire l'utilisateur courant directement depuis le contexte de sécurité Spring, sans passer par le `SecurityContextHolder`.

La **couche Service** contient la logique métier et les règles de gestion. Le `ConflictDetectionService` et le `ScheduleService` sont les services centraux du module de planification. Les services héritent de `BaseCrudService<T, ID, R>` qui fournit les opérations CRUD de base (`findAll`, `findByIdOrThrow`, `save`, `deleteWithCheck`), réduisant ainsi la duplication de code entre les modules.

La **couche Repository** assure la persistance des données via Spring Data JPA avec des requêtes dérivées et des `@Query` personnalisées. Les interfaces de repository n'ont pas besoin d'implémentations concrètes : Spring Data génère automatiquement les requêtes à partir des noms de méthodes (par exemple, `findByEmail` génère une requête `SELECT` avec clause `WHERE` sur l'email).

La **couche Entité** modélise les objets du domaine avec les annotations JPA (`@Entity`, `@OneToMany`, `@ManyToOne`) et les relations entre tables. L'héritage des utilisateurs est implémenté via la stratégie `JOINED`, et les données flexibles (membres de jury, coefficients) sont stockées via `@ElementCollection`.

Le projet est organisé en modules fonctionnels : `auth/` (authentification JWT), `admin/` (configuration des entités académiques), `coordinator/` (cœur métier avec 11 sous-modules), `teacher/` (évaluations et indisponibilités), `student/` (consultation et documents), `notification/` (email et notifications internes), `common/` (configuration Spring Security, OpenAPI, audit, mappers, exceptions).

4.3.2 Le Frontend : SPA React

L'interface utilisateur est une application monopage avec les caractéristiques suivantes.

La **gestion d'état asynchrone** est assurée par **TanStack Query v5** pour la synchronisation avec l'API, la mise en cache et le rechargement automatique des données. Chaque module dispose de ses propres hooks de requête (par ex. `use-admin-queries`, `use-coordinator-queries`) qui encapsulent les appels API et gèrent automatiquement le cache, les rechargements et les invalidations. Le `QueryClient` est configuré avec un `staleTime` de 30 secondes et un nombre de `retry` limité à 1, pour éviter les rechargements excessifs tout en autorisant une récupération rapide après une erreur réseau.

Le **routing** utilise `React Router v7` avec des routes imbriquées. Les routes publiques (`/login`, `/forgot-password`, `/print/*`) sont en dehors du layout authentifié. Les routes protégées sont regroupées par rôle sous `<DashboardLayout>` avec des gardes `ProtectedRoute` qui vérifient le rôle de l'utilisateur. Les routes d'administration (hors configuration) sont en outre protégées par `SetupGuard`, qui vérifie que la configuration initiale du système a été effectuée (`setupCompleted` dans `GeneralSettings`).

Le **design system** repose sur `Tailwind CSS 4` avec des variables CSS, `shadcn/ui` pour les composants de base (boutons, modales, tableaux), et des abstractions applicatives dans le répertoire `src/components/ui/`. Chaque composant UI est un wrapper personnalisé qui encapsule les composants `Radix UI` sous-jacents avec les styles `Tailwind` de l'application.

Les **composants spécifiques** sont organisés par module fonctionnel dans le répertoire `src/components/`. Par exemple, le sous-répertoire `coordinator/` contient les composants `DefenseCalendar`, `DraggableJurySlot`, `JurySidebar`, `SlotRow`, `RoomSearchSelect`, etc. Cette organisation permet de localiser rapidement les composants liés à une fonctionnalité donnée.

La **validation locale** est assurée par le moteur de conflits `TypeScript` (`conflict-engine.ts`), une réplique fonctionnelle du moteur `Java`, permettant une validation instantanée sans aller-retour serveur. Les schémas de validation `Zod` (`validations.ts`) garantissent la conformité des données saisies par l'utilisateur avant envoi au serveur.

4.4 Implémentation des modules fonctionnels

4.4.1 Module Authentification et Sécurité

La sécurité repose sur un mécanisme *stateless* utilisant les **JSON Web Tokens (JWT)**. Le flux d'authentification est géré par `AuthService` côté serveur :

```
public LoginResponse login(LoginRequest request) {
    User user = userRepository.findByEmail(request.email())
        .orElseThrow(() -> new ResponseStatusException(
            HttpStatus.UNAUTHORIZED, "Identifiants invalides"));

    if (!passwordEncoder.matches(request.password(), user.getPassword())) {
        throw new ResponseStatusException(
            HttpStatus.UNAUTHORIZED, "Identifiants invalides");
    }

    if (!user.isActive()) {
        throw new ResponseStatusException(
            HttpStatus.FORBIDDEN, "Compte inactif");
    }

    String token = jwtTokenProvider.generateToken(
        String.valueOf(user.getId()), user.getRole().name());
    return new LoginResponse(userMapper.toDto(user), token, expiresAt);
}
```

Le token JWT est stocké dans le `localStorage` du navigateur sous la clé "token" et transmis dans le header `Authorization: Bearer <token>` de chaque requête API via un intercepteur central (`src/lib/api-core.ts`). Un cookie `HttpOnly` (`jwt_token`, `SameSite=Lax`, durée 2 heures) est également défini lors de la connexion, offrant une seconde voie d'authentification pour les navigateurs qui bloquent le `localStorage`. Le contrôle d'accès est renforcé côté serveur par Spring Security avec des annotations `@PreAuthorize` basées sur les rôles (`ROLE_ADMIN`, `ROLE_COORDINATOR`, `ROLE_TEACHER`, `ROLE_STUDENT`).

Le module gère également la réinitialisation du mot de passe : l'utilisateur demande un lien de réinitialisation via `/api/auth/forgot-password`, le système génère un token UUID avec une durée de validité de 1 heure, envoie un email contenant le lien (via Mailpit en développement ou un serveur SMTP en production), et l'utilisateur définit un nouveau mot de passe via `/api/auth/reset-password`. La validation de la force du mot de passe utilise la bibliothèque `zxcvbn`, qui évalue la résistance aux attaques par force brute en temps réel.

Le `JwtAuthFilter` (qui étend `OncePerRequestFilter`) intercepte chaque requête, extrait le token du header `Authorization` ou du cookie `jwt_token`, le valide via `JwtTokenProvider`, charge l'utilisateur correspondant depuis un cache (`UserCacheService`), et configure le `SecurityContextHolder` Spring avec les autorisations appropriées. Les utilisateurs désactivés (`isActive = false`) sont bloqués avec un code 403.

4.4.2 Le Defense Designer : Planification Interactive

Le module de planification est l'élément le plus complexe du système. Il implémente une interface de type « canevas » permettant au coordinateur d'organiser les soutenances visuellement. L'architecture repose sur trois composants principaux : une barre latérale listant les jurys non planifiés, un calendrier interactif organisant les créneaux par salle et par date, et un panneau de détails affichant les informations du jury sélectionné.

L'interaction repose sur `@dnd-kit/core`, qui permet le glisser-déposer des jurys depuis la barre

latérale vers les cellules du calendrier. Chaque cellule représente un couple (date, créneau horaire) pour une salle sélectionnée. Le hook `useDefenseSchedule` orchestre l'ensemble du cycle de vie. Lors du dépôt d'un jury sur un créneau, le moteur de conflits local est invoqué pour valider l'opération. En cas d'erreur, le dépôt est annulé et un toast explicatif (avec suggestion de résolution) est affiché.

Les modifications du planning sont stockées dans un état React local (`useState`) avec la clé `juryId` et ne sont synchronisées avec le serveur qu'au moment de la sauvegarde explicite. Cette approche d'**état local optimiste** réduit drastiquement le nombre d'appels API (un seul appel de sauvegarde au lieu d'un appel par dépôt) et améliore la fluidité perçue.

L'algorithme d'**auto-génération** parcourt itérativement les salles, les jours et les créneaux pour proposer un planning initial. Pour chaque jury non planifié, l'algorithme cherche le premier créneau disponible en vérifiant les contraintes de capacité de salle et les conflits potentiels. Le `ScheduleService.autoGenerate` côté serveur implémente la même logique avec validation des contraintes de capacité, garantissant que le planning généré est valide avant persistance.

Le listing 4.4.2 montre le cœur de la logique de validation lors du drag-and-drop :

```
const handleDragEnd = (event: DragEndEvent) => {
  const { active, over } = event;
  setActiveJuryId(null);

  if (over && selectedRoomId) {
    const juryId = active.id as string;
    const [date, time] = (over.id as string).split("|");
    const jury = juries.find((j) => j.id === juryId);
    if (!jury) return;

    const slotKey = `${date}|${selectedRoomId}|${time}`;
    const context = buildConflictContext(
      schedule, juries, rooms, projects,
      teachers, unavailabilities, currentSession, timeSlots
    );

    const result = validateSlotAssignment(jury.projectId, slotKey, context);

    if (!result.isValid) {
      const error = result.issues.find(i => i.severity === "error")
        || result.issues[0];
      if (error.suggestedResolution) {
        toast.error(`${error.message} ${error.suggestedResolution}`,
          { duration: 5000 });
      } else {
        toast.error(error.message);
      }
      return; // Annulation du dépôt
    }

    setSchedule((prev) => ({
      ...prev,
      [juryId]: { roomId: selectedRoomId, date, time },
    }));
  }
}
```

```

    });
    toast.success("Positionné avec succès");
  }
};

```

4.4.3 Le Moteur de Détection de Conflits

L'intégrité du planning est garantie par le `ConflictDetectionService` qui exécute systématiquement 8 vérifications lors de chaque enregistrement. Le listing 4.4.3 montre la méthode centrale `runAllChecks` et l'une des vérifications les plus critiques :

```

private List<ConflictDetailResponse> runAllChecks(
    Map<String, ConflictSlot> schedule, String defenseSessionId) {
    List<ConflictDetailResponse> conflicts = new ArrayList<>();

    conflicts.addAll(checkProjectAlreadyScheduled(schedule)); // RG6
    conflicts.addAll(checkSlotOccupied(schedule));           // RG2
    conflicts.addAll(checkRoomCapacity(schedule));           // RG3
    conflicts.addAll(checkDateOutOfBounds(schedule, defenseSessionId)); //
    ↪ RG4
    conflicts.addAll(checkTeacherDoubleBooked(schedule));    // RG1
    conflicts.addAll(checkSupervisorConflict(schedule));      // RG5
    conflicts.addAll(checkBreakInterval(schedule, defenseSessionId)); // RG7
    conflicts.addAll(checkTeacherUnavailable(schedule));      // RG8

    return conflicts;
}

private List<ConflictDetailResponse> checkTeacherDoubleBooked(
    Map<String, ConflictSlot> schedule) {
    List<ConflictDetailResponse> conflicts = new ArrayList<>();
    Map<String, String> dateTeacherSlot = new HashMap<>();

    for (Map.Entry<String, ConflictSlot> entry : schedule.entrySet()) {
        String slotId = entry.getKey();
        ConflictSlot data = entry.getValue();
        String projectId = data.getProjectId();
        String date = data.getDate();
        if (projectId == null || date == null) continue;

        Set<String> teacherIds = getJuryTeacherIds(projectId);
        for (String tid : teacherIds) {
            String key = date + "|" + tid;
            if (dateTeacherSlot.containsKey(key)) {
                conflicts.add(createConflict(
                    "teacher_double_booked", "error",
                    "Un enseignant est déjà assigné à un autre projet le "
                    + date, slotId,
                    "Vérifiez la disponibilité des enseignants"));
            } else {
                dateTeacherSlot.put(key, slotId);
            }
        }
    }
}

```

```

    }
  }
}
return conflicts;
}

```

Le même moteur est dupliqué côté client en TypeScript (`conflict-engine.ts`). La fonction `validateSlotAssignment` implémente les mêmes 8 vérifications avec des messages d'erreur en français et des suggestions de résolution intelligentes. Le listing 4.4.3 montre la structure de la validation côté client :

```

export function validateSlotAssignment(
  projectId: string,
  slot: string,
  context: ConflictContext,
): { isValid: boolean; issues: ConflictIssue[] } {
  const issues: ConflictIssue[] = [];
  const [date, roomId, time] = slot.split("|");

  // RG2 - Créneau déjà occupé
  if (context.schedule[slot]) {
    issues.push({
      type: "slot_occupied", severity: "error",
      message: "Ce créneau est déjà occupé.", slot,
      suggestedResolution: getSmartSuggestions(
        projectId, date, roomId, time, context, "slot_occupied"
      ) || "Choisissez un autre créneau horaire ou une autre salle.",
    });
  }

  // RG3 - Capacité de salle
  const room = context.rooms[roomId];
  const project = context.projects[projectId];
  if (room && project && project.studentIds.length > room.capacity) {
    issues.push({
      type: "room_capacity", severity: "error",
      message: `La salle "${room.name}" a une capacité de ${room.capacity}
        mais le projet a ${project.studentIds.length} étudiants.`,
      slot,
      suggestedResolution: getSmartSuggestions(
        projectId, date, roomId, time, context, "room_capacity"
      ) || `Utilisez une salle avec capacite >= ${project.studentIds.length}`,
    });
  }
  // ... RG1, RG4, RG5, RG6, RG7, RG8
}

```

Le moteur client inclut également une fonction `getSmartSuggestions` qui analyse le contexte pour proposer des alternatives pertinentes (salles libres, créneaux disponibles) lorsqu'un conflit est détecté, améliorant ainsi l'expérience utilisateur. Cette fonction parcourt les salles disponibles pour la date et le créneau conflictuel, filtre par capacité, et retourne la première alternative trouvée

accompagnée d'un message d'explication.

4.4.4 Module d'Évaluation et Notation

Le module d'évaluation gère la saisie des notes par les membres du jury, le calcul des moyennes pondérées et le suivi de l'avancement des évaluations. La particularité du système réside dans la **pondération dynamique** : chaque session de soutenance définit des coefficients par rôle de jury (Président, Rapporteur, Examineur – chacun avec un coefficient configurable) stockés dans la table `defense_session_coefficients`.

L'enseignant accède à la page d'évaluations (`/teacher/evaluations`) qui affiche la liste des soutenances auxquelles il est assigné en tant que membre de jury. Pour chaque soutenance, il peut saisir un score (sur 20) et un commentaire. Le statut de l'évaluation évolue de `PENDING` (en attente de saisie) à `SUBMITTED` (note soumise). Une fois la note soumise, elle ne peut plus être modifiée, garantissant l'intégrité du processus d'évaluation.

Le calcul de la moyenne pondérée est effectué par le service `CoordinatorGradeService` via la méthode `getWeightedAverage`, qui agrège les scores individuels de chaque membre et les pondère selon les coefficients de la session. Le listing suivant illustre ce calcul :

```
public Double getWeightedAverage(Long defenseId, Long sessionId) {
    List<Evaluation> evals = evaluationRepository
        .findByDefenseIdAndStatus(defenseId, EvaluationStatus.SUBMITTED);
    Map<String, Integer> coefficients = defenseSessionRepository
        .findById(sessionId).get().getEvaluationCoefficients();

    double totalWeight = 0;
    double weightedSum = 0;
    for (Evaluation eval : evals) {
        Integer coeff = coefficients.getOrDefault(eval.getRole(), 0);
        weightedSum += eval.getScore() * coeff;
        totalWeight += coeff;
    }
    return totalWeight > 0 ? weightedSum / totalWeight : 0.0;
}
```

Le coordinateur peut consulter l'avancement des évaluations via la page des notes, qui affiche pour chaque soutenance le nombre d'évaluations soumises sur le nombre total attendu, avec une barre de progression visuelle. Cette vue permet d'identifier rapidement les soutenances pour lesquelles toutes les notes n'ont pas encore été saisies.

4.4.5 Génération Documentaire et Stratégie d'Impression

Plutôt que de générer des PDF lourds côté serveur (ce qui nécessiterait une bibliothèque comme Puppeteer ou iText), le système adopte une stratégie de **rendu client optimisé** exploitant les fonctionnalités natives du navigateur. Cinq types de documents sont supportés, chacun implémenté comme un composant React dédié optimisé pour l'impression A4.

La **fiche d'évaluation** (`PrintEvaluationSheet`) présente les informations du projet, du jury et de l'étudiant, avec des espaces dédiés à la saisie des notes et des observations par chaque membre du jury. La **liste de présence** (`PrintAttendanceList`) recense l'ensemble des soutenances d'une journée avec les noms des étudiants, des encadrants et des membres du jury,

permettant au coordinateur de vérifier la présence physique de chaque acteur. La **convocation jury** (`PrintJuryConvocation`) est une lettre officielle adressée à chaque membre du jury, mentionnant la date, l'heure, la salle et le projet à évaluer. Le **planning imprimable** (`PrintDefenseSchedule`) présente le planning complet sous forme de tableau, organisé par salle et par créneau horaire. Enfin, le **procès-verbal** (`PrintProcesVerbal`) documente le déroulement de la soutenance avec les notes finales, la décision du jury (Admis ou Ajourné) et les signatures.

La stratégie CSS `@media print` masque les éléments d'interface (menus, boutons, sidebar) et force le format A4 avec des marges précises lors de l'appel à `window.print()`. Le dialogue d'impression du navigateur permet à l'utilisateur d'enregistrer le document en PDF, offrant ainsi une génération de PDF sans aucune infrastructure serveur dédiée. Le `DocumentController` côté serveur fournit les données structurées (JSON) pour chaque type de document via des endpoints REST dédiés, et le `DocumentDataService` assemble les informations à partir des entités JPA (projets, jurys, créneaux, évaluations).

4.4.6 Système de Notifications

Le système de notifications internes permet au système de communiquer des informations importantes aux utilisateurs sans dépendre d'un canal externe (email). Chaque notification est une entité `AppNotification` stockée en base de données avec un type (SUCCESS, WARNING, ERROR, INFO), un titre, un message, un horodatage, un statut de lecture et un lien d'action optionnel.

Les notifications sont créées automatiquement par les services métier lors d'événements significatifs : création d'une soutenance, changement de statut d'une session, détection de conflit, soumission d'une évaluation, etc. Le frontend interroge le serveur toutes les 30 secondes via le hook `useNotifications` (configurable via `NOTIFICATION_POLL_INTERVAL`) pour récupérer les nouvelles notifications. Le composant `NotificationBadge` dans la sidebar affiche le nombre de notifications non lues sous forme de badge rouge.

L'endpoint `POST /api/notifications/{id}/send-email` permet aux administrateurs et coordinateurs de déclencher l'envoi d'un email associé à une notification, utilisant la configuration SMTP définie dans `EmailConfig`. En mode développement, les emails sont dirigés vers Mailpit, qui offre une interface web (<http://localhost:8025>) permettant de visualiser les emails envoyés sans nécessiter de serveur SMTP réel.

4.4.7 Système d'Audit

La traçabilité des actions est assurée par un système d'audit basé sur la programmation orientée aspect (AOP). L'annotation `@Audited(action="CREATE", entity="Department")` placée sur les méthodes de service déclenche automatiquement l'enregistrement d'une entrée dans la table `audit_log` via l'aspect `AuditAspect`.

L'aspect intercepte l'exécution de la méthode annotée, capture le résultat (succès ou échec), extrait l'identifiant de l'entité concernée (soit depuis les arguments de la méthode, soit depuis la valeur de retour via réflexion), et écrit un journal d'audit dans une transaction indépendante (`TransactionTemplate`) pour garantir l'enregistrement même en cas d'échec de la transaction métier.

Chaque entrée d'audit contient l'action effectuée (CREATE, UPDATE, DELETE), le type d'en-

tité concernée, l'identifiant de l'entité, l'email de l'utilisateur ayant effectué l'action, les détails de l'opération et l'horodatage. Le coordinateur d'administration peut consulter ces journaux via la page `/admin/audit-logs`, qui offre une vue paginée et filtrable de l'ensemble des actions effectuées dans le système.

4.4.8 Module de Configuration Administrative

Le module d'administration gère la configuration structurelle du système à travers quatre entités singleton (identifiant fixe `id=1L`) et plusieurs entités de référence. Les configurations singleton sont `GeneralSettings` (nom de l'institution, logo, fuseau horaire, statut de configuration initiale), `DefenseSettings` (horaires de début/fin, durée des soutenances, intervalle de pause, périodes de création de groupes), `EmailConfig` (serveur SMTP, identifiants, nom d'expéditeur) et `DocumentConfig` (taille maximale des fichiers, extensions autorisées, limite de versions).

Les entités de référence (`Grade`, `Level`, `Major`, `JuryRoleTemplate`) sont gérées via des interfaces CRUD standard avec recherche et pagination. Le composant `ConfigEntityManager` est un composant réutilisable qui fournit automatiquement les dialogues de création, modification et suppression pour toute entité de référence, réduisant ainsi la duplication de code entre les différentes pages de configuration.

Le `SetupGuard` implémente une logique de première utilisation : si `setupCompleted` est à `false`, l'utilisateur est redirigé vers la page de configuration (`/admin/config`) avant d'accéder à toute autre page d'administration. Cette mécanique garantit que les paramètres essentiels (nom de l'institution, durée des soutenances) sont configurés avant la première utilisation du système.

4.4.9 Fonctionnalités Étudiant et Enseignant

Le module étudiant offre trois fonctionnalités principales. La page `/student/group` permet de créer un groupe de projet, de rejoindre un groupe existant via un code d'invitation, et de consulter la composition actuelle du groupe. La page `/student/documents` affiche la liste des documents requis (convocation, fiche d'évaluation, etc.) avec leur statut (manquant, soumis, accepté) et permet de télécharger les convocations. Le hook `useStudentGroup` gère la logique de création et de jonction de groupe avec validation côté client.

Le module enseignant complète les fonctionnalités de jury. La page d'indisponibilités fournit un calendrier interactif permettant de déclarer les créneaux d'indisponibilité sur les prochaines semaines. Les indisponibilités sont enregistrées via l'endpoint dédié et prise en compte par le moteur de conflits (RG8). La page de planning personnel affiche le planning de l'enseignant, filtré pour ne montrer que les soutenances auxquelles il est assigné en tant que membre de jury.

4.5 Interface utilisateur et navigation

L'interface utilisateur est organisée en sections distinctes accessibles via une sidebar adaptative, chaque section correspondant à un rôle spécifique. Le tableau 11 présente la cartographie complète des routes de l'application :

Chemin	Page / Composant	Accès
/login	Connexion	Public
/verify-account	Activation de compte	Public
/forgot-password	Mot de passe oublié	Public
/reset-password	Réinitialisation MDP	Public
/admin	Dashboard Admin	Admin
/admin/config	Configuration système	Admin
/admin/departments	Gestion départements	Admin
/admin/rooms	Gestion salles	Admin
/admin/users/students	Gestion étudiants	Admin
/admin/users/teachers	Gestion enseignants	Admin
/admin/users/coordinators	Gestion coordinateurs	Admin
/admin/audit-logs	Journal d'audit	Admin
/coordinator	Dashboard Coordinateur	Coord.
/coordinator/schedule	Defense Designer	Coord.
/coordinator/projects	Projets & Groupes	Coord.
/coordinator/juries	Constitution jurys	Coord.
/coordinator/defense-sessions	Sessions de soutenance	Coord.
/coordinator/conflicts	Tableau des conflits	Coord.
/coordinator/grades	Suivi des notes	Coord.
/coordinator/documents	Documents	Coord.
/teacher	Dashboard Enseignant	Ens.
/teacher/schedule	Planning personnel	Ens.
/teacher/evaluations	Saisie des notes	Ens.
/teacher/unavailability	Indisponibilités	Ens.
/student	Dashboard Étudiant	Étud.
/student/group	Groupe de projet	Étud.
/student/documents	Documents & convocation	Étud.
/profile	Profil utilisateur	Tous
/notifications	Notifications	Tous
/print/evaluation-sheet	Fiche d'évaluation	Public
/print/attendance-list	Liste de présence	Public
/print/jury-convocation	Convocation jury	Public
/print/schedule	Planning imprimable	Public
/print/proces-verbal	Procès-verbal	Public

TABLE 11 – Cartographie complète des pages de l'application

Chaque route protégée est encapsulée dans un composant `ProtectedRoute` qui vérifie le rôle de l'utilisateur depuis le token JWT stocké localement. En cas de non-autorisation, l'utilisateur

est redirigé vers la page de connexion. Les routes d'administration (hors configuration) sont en outre protégées par `SetupGuard`, qui vérifie que la configuration initiale du système a été effectuée. Les routes d'impression (`/print/*`) sont les seules routes authentifiées en dehors du layout `DashboardLayout`, car elles nécessitent un rendu sans sidebar ni barre de navigation pour produire des documents propres à l'impression.

4.6 Bonnes pratiques adoptées

Le développement du système a été guidé par des bonnes pratiques industry visant à garantir la qualité, la maintenabilité et la reproductibilité du code.

4.6.1 Tests automatisés

La stratégie de tests suit la pyramide des tests : un grand nombre de tests unitaires à la base, un nombre modéré de tests d'intégration au milieu, et un nombre réduit de tests de bout en bout au sommet. Côté backend, 86 classes de test JUnit 5 + Mockito couvrent les services, les contrôleurs et les utilitaires, avec une couverture de 85% des lignes et 70% des branches mesurée par JaCoCo. Côté frontend, 91 fichiers de test Vitest + React Testing Library couvrent les pages, les hooks, les composants et les utilitaires, avec une stratégie de mocking basée sur MSW (Mock Service Worker) qui simule les réponses de l'API sans appel réseau. Les tests E2E Playwright (environ 65 tests) valident les parcours utilisateur complets contre l'API réelle, avec une authentification préalable pour les 4 rôles.

4.6.2 Hooks de pré-commit

Husky et lint-staged implémentent des hooks de pré-commit qui s'exécutent automatiquement avant chaque commit Git. Pour les fichiers TypeScript/TSX, ESLint et Prettier sont exécutés en parallèle. Pour les fichiers Java, le formatage est vérifié via SpotBugs, Checkstyle et PMD. Si l'une de ces vérifications échoue, le commit est bloqué, empêchant l'introduction de code non formaté ou contenant des erreurs de linting dans le dépôt. Cette barrière de qualité, combinée aux hooks post-commit qui exécutent les tests unitaires, constitue un filet de sécurité qui maintient la qualité du code à chaque étape du développement.

4.6.3 Conventions Git et GitHub

Les messages de commit suivent les conventions sémantiques : préfixes `feat:`, `fix:`, `chore:`, `docs:`, `test:`, suivis d'une description en anglais à l'impératif. Les pull requests documentent les changements apportés et lient les issues GitHub correspondantes. Les issues GitHub servent de support au suivi des tâches, avec des labels de priorité (`high`, `medium`, `low`) et de catégorie (`bug`, `feature`, `documentation`). Les releases GitHub documentent les versions majeures avec un changelog structuré.

4.6.4 Pipeline CI/CD

Le pipeline GitHub Actions automatise la construction, les tests et la synchronisation entre les dépôts. Trois workflows distincts s'exécutent à chaque push sur `main` : le workflow de l'API compile le code Java avec Maven, exécute les tests unitaires et vérifie la couverture de code. Le workflow de l'UI lance le build Vite et les tests Vitest. Le workflow E2E synchronise les sources, démarre les deux serveurs, et exécute les tests Playwright. La synchronisation entre les

dépôts est assurée par des scripts rsync qui copient les sources dans le dépôt E2E, garantissant que les tests d'intégration s'exécutent toujours sur la dernière version du code.

4.7 Difficultés rencontrées et solutions

Plusieurs difficultés techniques ont été rencontrées au cours du développement, nécessitant des solutions adaptées.

La principale difficulté a été la **synchronisation de l'état du planning** entre l'interface interactive (DnD) et la base de données. Le Defense Designer manipule un état local complexe (associations jury → créneau) qui doit être validé contre 8 règles de gestion avant persistance. La solution a consisté à utiliser un **état local optimiste** dans le hook `useDefenseSchedule`, qui ne synchronise le serveur qu'au moment de l'enregistrement final. Cette approche réduit drastiquement le nombre d'appels API (un seul appel de sauvegarde au lieu d'un appel par dépôt) et améliore la fluidité perçue.

Une seconde difficulté a été la **duplication du moteur de conflits** entre Java et TypeScript. Pour garantir la cohérence, les deux implémentations suivent exactement la même séquence de vérifications et les mêmes structures de données (projet, salle, enseignant, indisponibilité). Les tests unitaires des deux moteurs valident les mêmes scénarios, et les messages d'erreur sont maintenus de manière cohérente entre les deux implémentations.

Une troisième difficulté a été la **conception du système de templates de jurys**. Les départements ont des besoins différents en termes de composition de jury (certains exigent un Président, un Rapporteur et un Examineur, d'autres ajoutent un Co-encadrant). La solution a été d'implémenter des `JuryRoleTemplate` configurables par l'administrateur, chaque template définissant les rôles nécessaires, le nombre de membres par rôle et le coefficient associé. Cette flexibilité permet au système de s'adapter à différents contextes sans modification du code.

La quatrième difficulté concernait la **compatibilité inter-navigateurs des composants d'impression**. Les feuilles de style `@media print` se comportent différemment selon les navigateurs (Chrome, Firefox, Safari), notamment pour le rendu des tableaux et la gestion des sauts de page. La solution a été d'adapter les composants Print pour qu'ils fonctionnent de manière optimale sur Chrome (le navigateur le plus utilisé), tout en restant fonctionnels sur les autres navigateurs.

4.8 Conclusion

Ce chapitre a présenté la traduction technique de la conception. L'utilisation d'une architecture découplée (API REST + SPA React), d'un moteur de conflits rigoureux implémenté en Java et dupliqué en TypeScript pour la réactivité, et d'une interface utilisateur moderne basée sur Tailwind CSS et shadcn/ui, a permis de répondre aux exigences de fiabilité et d'ergonomie fixées. Les modules de notifications, d'audit et de configuration administrative complètent le système en offrant les fonctionnalités transverses nécessaires à un usage productif. Les bonnes pratiques adoptées (tests automatisés, hooks de pré-commit, conventions Git, pipeline CI/CD) garantissent la maintenabilité du code à long terme. Le système est désormais prêt pour la phase de validation qui sera détaillée au chapitre suivant.

5 Stratégie de test et validation

5.1 Introduction

Ce chapitre présente la démarche de validation mise en œuvre pour garantir la fiabilité du système de gestion des soutenances. Dans un domaine où les données manipulées – notes, jurys, plannings – ont une incidence directe sur le parcours académique des étudiants, la qualité logicielle n'est pas un luxe mais une exigence fondamentale. Une erreur de planification peut priver un étudiant de sa soutenance, une erreur de calcul de moyenne peut modifier une décision de jury, et un bug de sécurité peut exposer des données confidentielles. C'est pourquoi nous avons adopté une stratégie de test pyramidale multi-couche, combinant des tests unitaires, d'intégration et des tests de bout en bout (E2E), permettant de détecter les régressions à différents niveaux de l'application avec un rapport coût-efficacité optimal.

5.2 Stratégie de test

La stratégie repose sur la validation progressive de chaque couche du système, depuis la logique métier isolée jusqu'au parcours utilisateur complet. Cette approche en pyramide – beaucoup de tests unitaires rapides à la base, un nombre modéré de tests d'intégration au milieu, et un nombre réduit de tests E2E au sommet – offre le meilleur compromis entre couverture, vitesse d'exécution et coût de maintenance.

5.2.1 Tests unitaires et d'intégration (Backend)

Le backend Java contient **86 classes de test** réparties dans le répertoire de tests du backend, calqué sur la structure du code source. Chaque module fonctionnel dispose de son propre ensemble de tests : `auth/`, `admin/`, `coordinator/`, `teacher/`, `student/`, `notification/` et `common/`.

Les tests sont réalisés avec **JUnit 5** et **Mockito**, et se concentrent sur deux axes complémentaires. Les tests unitaires des services isolent chaque service métier avec mock des dépendances (repositories, autres services), permettant de valider la logique de calcul, les règles de gestion et la gestion des erreurs sans aucune interaction avec la base de données. Les tests d'intégration des contrôleurs, réalisés via `@WebMvcTest`, testent les endpoints REST avec le contexte Spring complet, validant la conformité des contrats (statuts HTTP, format JSON), la sérialisation des DTOs, la gestion des erreurs (exceptions métier, validation échouée) et la sécurité (accès refusé pour les rôles non autorisés).

Parmi les classes de test notables, `ConflictDetectionServiceTest` constitue la suite la plus importante. Elle valide exhaustivement chacune des 8 règles de gestion (RG1–RG8) avec des jeux de données spécifiques couvrant les cas nominaux (pas de conflit), les cas limites (conflit exact à la frontière d'un créneau), et les cas d'erreur (conflit multiple). Chaque test suit le pattern Arrange–Act–Assert : préparation des données de test (projets, salles, enseignants, indisponibilités), exécution de la méthode de vérification, et vérification du résultat (liste de conflits vide ou non vide selon le cas testé).

Les autres classes de test couvrent l'ensemble des modules : `AuthServiceTest` valide le cycle de vie de l'authentification (login, activation, reset), `ScheduleServiceTest` teste la logique de planification et d'auto-génération, `JuryServiceTest` teste la constitution des jurys et de la gestion des membres, `CoordinatorDefenseSessionServiceTest` teste le cycle de vie des sessions, `DocumentDataServiceTest` teste la génération des données

documentaires, `NotificationServiceTest` teste le système de notifications internes, et `StudentDocumentServiceTest` teste la logique d'accès aux documents côté étudiant.

La couverture de code est monitorée via **JaCoCo (v0.8.12)** avec les objectifs suivants, vérifiés à chaque construction Maven (`mvn verify`) : couverture minimale de 85% sur les lignes de code et 70% sur les branches conditionnelles. Les exclusions portent sur les DTOs, les entités JPA (structures de données pures sans logique), les classes de configuration Spring et le code de seed (peuplement initial de la base de démonstration). Le rapport de couverture est généré en HTML et en XML, ce dernier étant consommé par les outils de CI pour vérifier les seuils automatiquement.

Les vérifications de qualité statique complètent les tests. **Checkstyle** vérifie la conformité du code aux conventions de codage Java (nommage, indentation, longueur des lignes, imports). **PMD** détecte les problèmes de qualité de code (code mort, duplication, complexité cyclomatique excessive, variables non utilisées). **SpotBugs** analyse le code bytecode pour identifier les bugs potentiels (pointeurs nuls, fuites de ressources, problèmes de thread safety). Ces trois outils sont intégrés au build Maven via les plugins `maven-checkstyle-plugin`, `maven-pmd-plugin` et `spotbugs-maven-plugin`, et s'exécutent automatiquement lors de chaque `mvn verify`.

5.2.2 Tests de l'interface utilisateur (Frontend)

Le frontend TypeScript utilise **Vitest (v4.1.7)** avec **React Testing Library** et **jsdom** pour l'environnement DOM simulé. La suite de tests frontend comprend **91 fichiers de test** (86 `*.test.tsx` + 5 `*.test.ts`), organisés dans le répertoire `src/test/` qui reproduit la structure de `src/`.

L'architecture de mocking repose sur **MSW (Mock Service Worker) v3**, qui intercepte les appels réseau au niveau du service worker du navigateur simulé. Le serveur MSW est configuré dans le fichier de setup des mocks et démarre automatiquement avant chaque test via le setup global. Les handlers de mock fournissent des réponses simulées pour l'ensemble des endpoints API (auth, admin, coordinator, teacher, student). Le module registry expose des helpers de haut niveau : `mockJson` pour les réponses simples, `mockPaginated` pour les réponses paginées, `mockCrud` pour les opérations CRUD complètes, et `mockEcho` pour retourner le corps de la requête (utile pour tester les mutations).

Le composant utilitaire `src/test/utils.tsx` fournit `renderWithProviders`, une fonction qui enveloppe tout composant testé dans les providers nécessaires : `QueryClientProvider` (TanStack Query), `MemoryRouter` (React Router), `AuthProvider` (contexte d'authentification) et `TooltipProvider` (shadcn/ui). Cette abstraction élimine la duplication de configuration entre les tests et garantit un environnement de test cohérent.

Les tests couvrent cinq catégories de composants. Les hooks personnalisés sont testés via des composants « harness » qui les appellent dans un contexte React contrôlé. Les composants UI (`DefenseCalendar`, `DraggableJurySlot`, `JurySidebar`, `AvailabilityCalendar`, `ProjectDialog`) sont testés pour leur rendu, leurs interactions et leur état. La validation (`conflict-engine.test.ts`) est testée unitairement pour garantir la cohérence avec le moteur Java. Le routage (`ProtectedRoute`, `SetupGuard`) est testé pour les redirections selon les rôles. Enfin, chaque page de l'application dispose d'un test de rendu validant la présence des éléments clés.

5.2.3 Tests de bout en bout (E2E)

La validation finale du système est confiée à **Playwright (v1.52)**, qui simule des interactions réelles dans un navigateur Chromium. Contrairement aux tests unitaires et d'intégration qui tournent contre des mocks, les tests E2E s'exécutent contre l'API Spring Boot réelle et la base de données H2, garantissant une fidélité maximale aux comportements de production.

L'architecture repose sur le pattern **setup-project** de Playwright. Le fichier `auth.setup.ts` s'exécute une seule fois avant tous les tests et authentifie les 4 rôles (admin, coordinateur, enseignant, étudiant) en remplissant le formulaire de connexion et en soumettant les credentials. Le contexte de navigateur résultant (cookies JWT, localStorage) est sérialisé dans des fichiers JSON (`.auth/admin.json`, etc.) et réutilisé via l'option `storageState` de chaque bloc `test.describe`. Cette approche élimine la phase de connexion pour chaque test, réduisant le temps d'exécution et la fragilité liée aux changements d'interface de connexion.

Les tests E2E sont organisés en 8 fichiers de test, chacun ciblant un rôle ou un flux spécifique : `auth.spec.ts` teste le cycle de connexion (succès, échec, déconnexion, reset MDP, activation), `admin.spec.ts` teste le CRUD des départements, salles et utilisateurs, `coordinator.spec.ts` teste la création et transition des sessions ainsi que la gestion des projets, `teacher.spec.ts` teste la saisie des indisponibilités et des évaluations, `student.spec.ts` teste la consultation du groupe et des documents, `audit-logs.spec.ts` vérifie l'accès aux journaux d'audit par rôle sur l'ensemble des routes, et `system.spec.ts` teste les notifications et le responsive design.

La configuration Playwright utilise un worker unique (`workers: 1`) et le parallélisme désactivé (`fullyParallel: false`), car les tests s'exécutent contre une base de données H2 partagée dont l'état est modifié par chaque test. Les tentatives de rejeu sont désactivées en développement (0) et activées en CI (2). Les traces et captures d'écran ne sont conservées qu'en cas d'échec, pour faciliter le diagnostic sans encombrer l'espace disque.

Le pipeline CI (`.github/workflows/e2e-tests.yml`) automatise l'exécution : les jobs `build-api` et `build-ui` construisent les artefacts en parallèle, puis le job `e2e` les télécharge, démarre les deux serveurs avec health-check loops, exécute les tests Playwright, et génère le rapport HTML. Ce pipeline s'exécute à chaque fusion sur la branche `main` et garantit que les tests d'intégration s'exécutent toujours sur la dernière version du code.

5.3 Scénarios de validation et résultats

Le tableau 12 présente dix scénarios de validation couvrant les fonctionnalités critiques du système, du niveau unitaire jusqu'au bout en bout. Chaque scénario décrit les étapes d'exécution, les résultats attendus et le verdict de validation.

Niveau	Fonctionnalité	Scénario de test	Résultat
Unitaire	Détection conflits RG1–RG8	Chevauchement enseignant, capacité salle, pause minimale, indisponibilité	Validé (8/8)
Unitaire	Calcul moyenne pondérée	Coefficients 40/35/25, notes 14/16/12, vérification du résultat	Validé
Intégration	API Authentification	Login success/failed, JWT généré, rôle correct, compte inactif bloqué	Validé
Intégration	API Planification	Save schedule, load schedule, auto-generate, conflict detection serveur	Validé
UI	Defense Designer	Drag jury vers cellule, validation locale toast erreur, toast succès, sauvegarde	Validé
UI	Impression documents	Rendu A4 convocation via @media print, 5 types de documents	Validé
UI	ProtectedRoute	Redirection student vers /login si accès /admin, toast session expirée	Validé
E2E	Flux Admin	Créer/modifier/supprimer département, créer étudiant, créer salle	Validé
E2E	Flux Coordinateur	Créer session, transition draft→active, créer projet, accéder au designer	Validé
E2E	Flux Enseignant	Déclarer indisponibilité, soumettre évaluation, consulter planning	Validé

TABLE 12 – Matrice de validation des fonctionnalités critiques

5.4 Métriques de qualité

Le tableau 13 résume l'ensemble des métriques de qualité mesurées lors du développement du système. Ces chiffres sont vérifiés automatiquement à chaque construction et exportés dans les rapports CI.

Métrique	Valeur	Outil
Classes de test backend (JUnit 5)	86	Maven Surefire
Fichiers de test frontend (Vitest)	91	Vitest + v8 coverage
Tests E2E (Playwright)	≈ 65	Playwright
Couverture lignes backend	85%	JaCoCo
Couverture branches backend	70%	JaCoCo
Contrôleurs REST	37	Comptage manuel
Endpoints API	≈ 80	Swagger UI
Composants React (src/components)	39+	Comptage manuel
Pages de l'application (routes)	33	Route mapping
Hooks TanStack Query	25+	Fichiers use-*.queries.ts

TABLE 13 – Métriques de qualité du système

5.5 Évaluation globale et limites

Le système répond aux exigences fonctionnelles du cahier des charges. La détection automatique des conflits réduit drastiquement le temps de planification et élimine les erreurs humaines. L'intégration des tests dans le pipeline CI garantit la non-régression des fonctionnalités critiques lors de chaque modification du code. La pyramide de tests – 86 tests unitaires Java, 91 tests frontend TypeScript, 65 tests E2E Playwright – offre une couverture à chaque niveau de l'application.

Cependant, certaines limites subsistent et méritent d'être documentées. La couverture E2E, bien que fonctionnelle, ne couvre pas encore l'intégralité des cas limites et des parcours utilisateurs complexes, notamment les scénarios de rattrapage et les flux d'erreur (serveur indisponible, token expiré). La génération de PDF repose sur le dialogue d'impression du navigateur, ce qui peut varier légèrement selon le système d'exploitation et la version du navigateur – une évolution vers une génération côté serveur (via Puppeteer ou iText) est envisagée pour garantir un rendu identique.

L'algorithme d'auto-génération du planning utilise une approche gloutonne (greedy) qui assigne les jurys au premier créneau disponible, sans recherche d'optimum global. Pour un grand nombre de projets (> 100), cette approche peut produire un sous-optimalisme dans l'occupation des salles. Des algorithmes d'optimisation plus sophistiqués (programmation par contraintes, recuit simulé) pourraient être envisagés dans une version future.

Le système ne supporte actuellement ni les mises à jour en temps réel (WebSocket) ni le mode hors ligne. Les notifications nécessitent un rechargement périodique (toutes les 30 secondes), et les données du planning ne sont accessibles qu'en ligne. Enfin, l'utilisation de H2 en base de données de développement, bien que pratique pour les tests et le seed data, ne reproduit pas fidèlement toutes les fonctionnalités de MySQL (notamment les requêtes géospatiales et les index composites), ce qui peut masquer des problèmes de performance en production.

5.6 Conclusion

La stratégie de test multi-couche a permis d'aboutir à une application stable et fiable. La combinaison de 86 tests unitaires Java garantissant 85% de couverture, 91 tests frontend TypeScript

validant l'interface utilisateur, et environ 65 tests E2E Playwright couvrant les parcours principaux, assure que le système est prêt pour un déploiement en environnement académique. L'intégration des outils de qualité (JaCoCo, Checkstyle, PMD, SpotBugs, ESLint, Prettier) dans le pipeline Maven et les hooks Husky garantit la pérennité de la qualité du code au fil des évolutions futures.

Conclusion Générale

Ce projet de fin d'études a permis la conception et le développement d'une solution complète pour la gestion des soutenances et des jurys au sein d'un établissement universitaire. Partant des constats d'inefficacité des processus manuels existants – tableurs Excel, échanges d'e-mails, documents papier – nous avons développé une application web moderne et intégrée, articulée autour d'une architecture Spring Boot (backend Java 17) et React (frontend TypeScript 6, Tailwind 4), répondant aux problématiques concrètes de coordination, d'automatisation des flux documentaires et de gestion des contraintes liées à l'organisation des soutenances universitaires.

Les objectifs initiaux ont été atteints avec succès. L'automatisation de la planification, via un designer de défenses interactif avec détection automatique de conflits basée sur huit règles de gestion (RG1–RG8), a remplacé le processus manuel chronophage et sujet aux erreurs. L'intégration d'un moteur d'auto-génération de planning a permis de réduire significativement le temps de constitution des plannings de soutenance. La production documentaire automatisée – convocations, fiches d'évaluation, procès-verbaux – élimine la saisie manuelle répétitive et garantit l'uniformité des formats. Le module d'évaluation, avec calcul automatique des moyennes pondérées et gestion formelle des délibérations, assure la fiabilité et la traçabilité des résultats académiques. Le contrôle granulaire des accès, basé sur quatre rôles (administrateur, coordinateur, enseignant, étudiant) avec JWT et tokens d'activation, garantit la sécurité des données sensibles.

Le système a été validé via une stratégie de test multi-couche comprenant 86 classes de test backend (JUnit 5 + Mockito) avec 85% de couverture sur les lignes et 70% sur les branches, 91 fichiers de test frontend (Vitest + React Testing Library) et environ 65 scénarios de test de bout en bout (Playwright) exécutés contre l'API Spring Boot réelle et la base de données H2. Les outils de qualité statique (JaCoCo, Checkstyle, PMD, SpotBugs, ESLint, Prettier) intégrés au pipeline Maven et aux hooks Husky garantissent la pérennité de la qualité du code. Ce système constitue un levier d'amélioration opérationnelle significatif pour les établissements universitaires, permettant de réduire la charge administrative tout en augmentant la fiabilité et la traçabilité des processus organisationnels.

Les perspectives d'évolution sont nombreuses et pertinentes. L'intégration d'un moteur de rendu PDF côté serveur (via Puppeteer ou iText) permettrait d'automatiser la génération et l'envoi des convocations par email, éliminant ainsi le besoin d'impression manuelle. Le développement d'une application mobile dédiée offrirait des notifications en temps réel via les services push natifs, améliorant la réactivité des enseignants et des étudiants face aux changements de planning. L'optimisation des algorithmes d'auto-génération du planning, en remplaçant l'approche gloutonne actuelle par des techniques de programmation par contraintes ou de recuit simulé, permettrait de prendre en compte des contraintes de préférence plus fines des enseignants et d'optimiser l'occupation des salles. Enfin, l'intégration de WebSocket pour les mises à jour en temps réel et le développement d'un mode hors ligne progressive (PWA) constitueraient des améliorations significatives de l'expérience utilisateur.

Références

- [1] Spring Boot Documentation, version 3.4.6, <https://docs.spring.io/spring-boot/docs/3.4.6/reference/html/>
- [2] Spring Data JPA Documentation, <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
- [3] Spring Security Documentation, <https://docs.spring.io/spring-security/reference/>
- [4] Hibernate ORM Documentation, version 6.6, <https://docs.hibernate.org/hibernate/6.6/htmlguide/>
- [5] Java SE 17 Documentation, <https://docs.oracle.com/en/java/javase/17/>
- [6] React Documentation, version 19, <https://react.dev/reference/react>
- [7] React Router Documentation, version 7, <https://reactrouter.com/>
- [8] TanStack Query Documentation, version 5, <https://tanstack.com/query/latest>
- [9] Shadcn/ui Documentation, <https://ui.shadcn.com/>
- [10] Tailwind CSS Documentation, version 4, <https://tailwindcss.com/docs>
- [11] Vite Documentation, version 8, <https://vite.dev/guide/>
- [12] TypeScript Documentation, version 6, <https://www.typescriptlang.org/docs/>
- [13] Playwright Documentation, version 1.52, <https://playwright.dev/docs/intro>
- [14] Vitest Documentation, version 4.1, <https://vitest.dev/guide/>
- [15] Mock Service Worker (MSW) Documentation, version 3, <https://mswjs.io/>
- [16] RFC 7519 – JSON Web Token (JWT), <https://datatracker.ietf.org/doc/html/rfc7519>
- [17] MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- [18] H2 Database Engine Documentation, <https://h2database.com/html/main.html>
- [19] Pro Git Book, version 2, <https://git-scm.com/book/>
- [20] PlantUML Documentation, <https://plantuml.com/>
- [21] Unified Modeling Language (UML) Specification, version 2.5.1, <https://www.omg.org/spec/UML/2.5.1>
- [22] Manifesto for Agile Software Development, <https://agilemanifesto.org/>
- [23] Organisation LMD – Ministère de l'Enseignement Supérieur du Maroc, <https://www.gov.ma/>